Tower of Hanoi

Ahmmad Nur Swapnil (2005009) Al Muhit Muhtadi (2005013)
Ekramul Haque Amin (2005022)

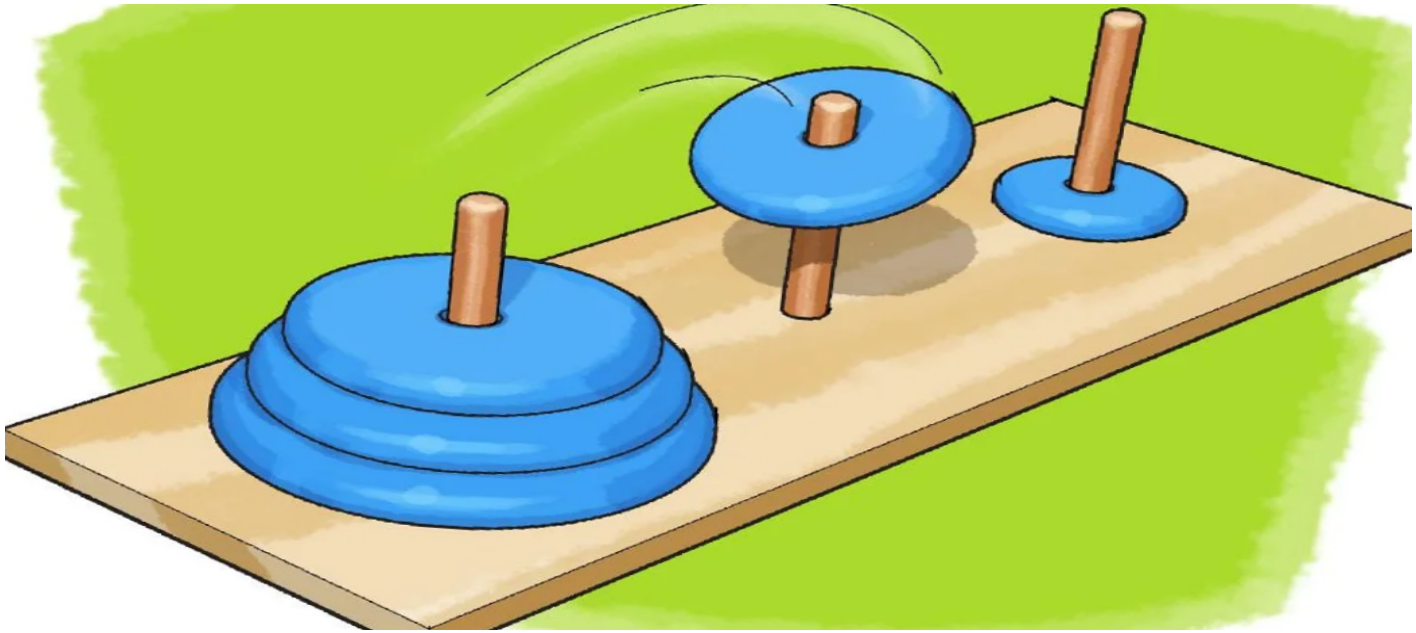
Department of CSE
Bangladesh University of Engineering and Technology

February 13, 2024

Introduction

What is Tower of Hanoi?

The Tower of Hanoi is a mathematical game or puzzle consisting of three poles and a number of disks of various diameters. The puzzle begins with the disks stacked on one pole in order of decreasing size. The objective of the puzzle is to move the entire stack to one of the other poles, obeying the rules.



Introduction



Origin

The puzzle was invented by the French mathematician **Édouard Lucas** in 1889. It was first published in a booklet and later included in a collection of his math puzzles called *Récréations mathématiques* after his death.

Introduction

Legend

In an ancient Asian legend, attributed to various regions like Tibet, Vietnam, or India, monks engage in a puzzling task involving 64 disks. According to the tale, these monks believe that once they successfully transfer all the disks from one pole to another following strict rules, it will start the END of the world!!

Introduction

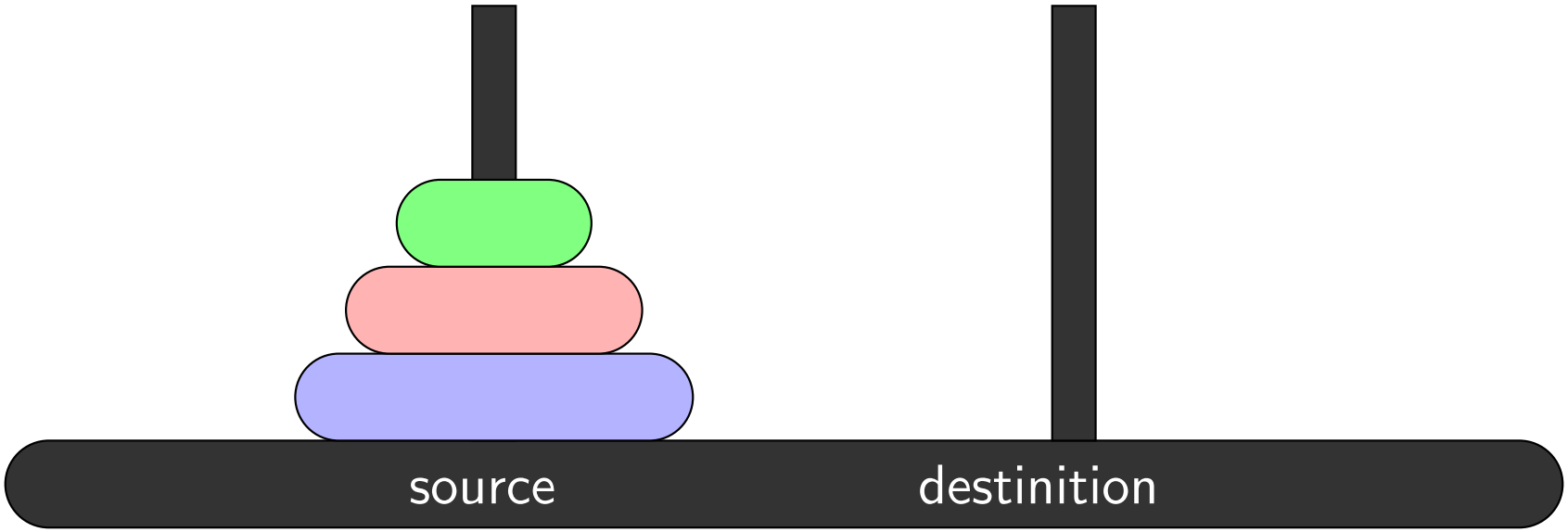
If the monks are correct,
Should we be panicking in the streets?!



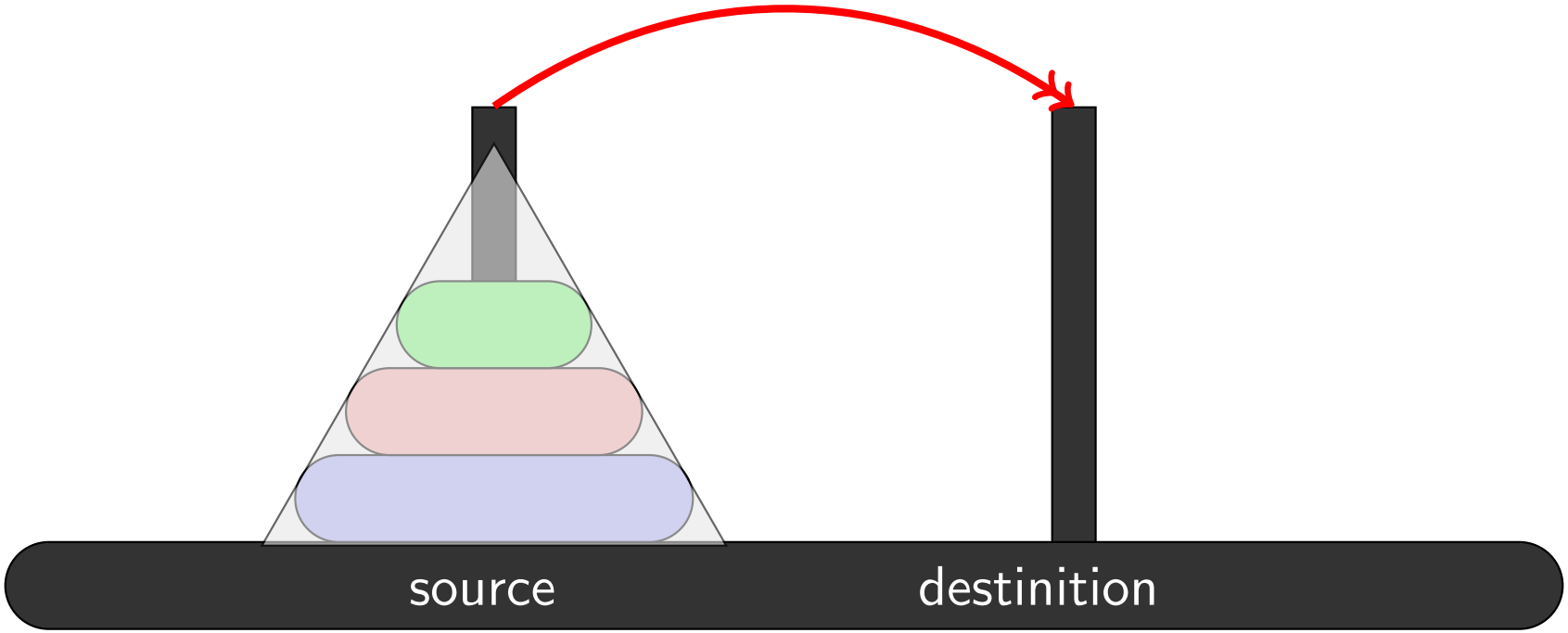
Let's find out .....

Tower of Hanoi

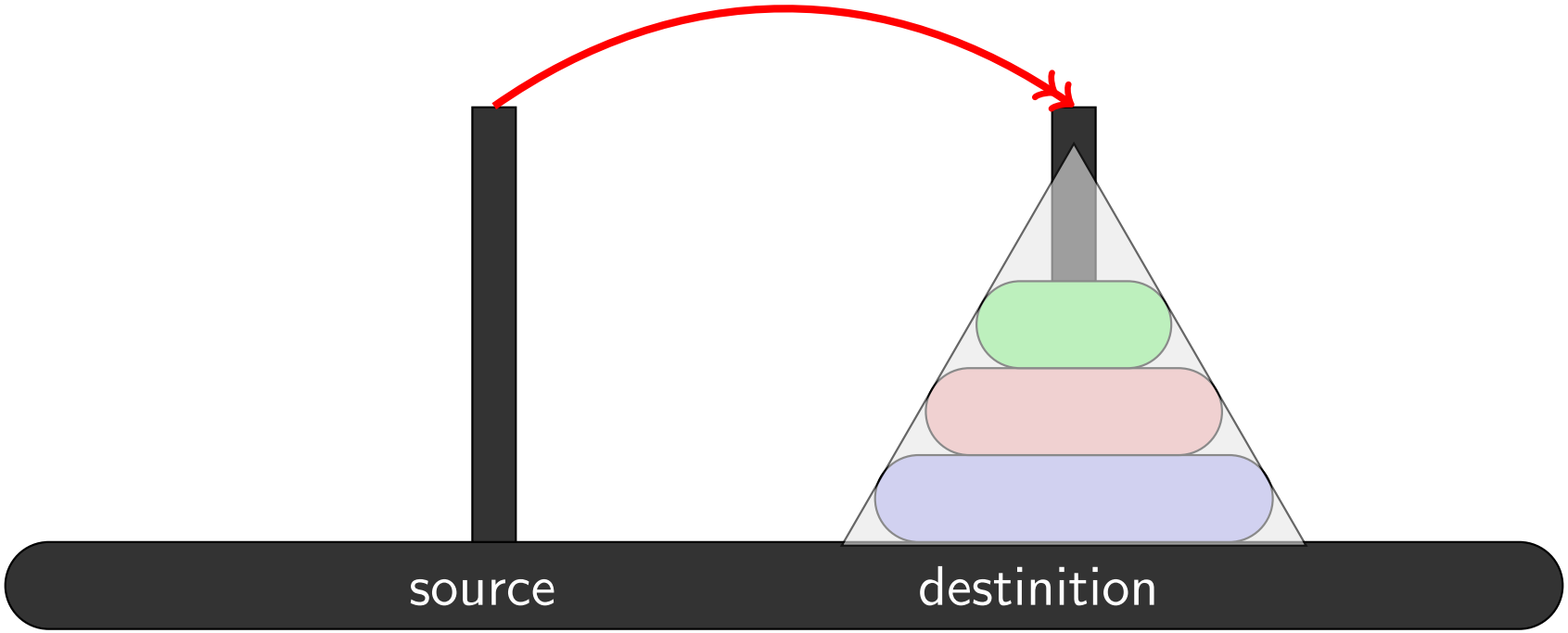
Lets solve a smaller version of the problem We will move these 3 discs to the destination.
Sounds easy,right?



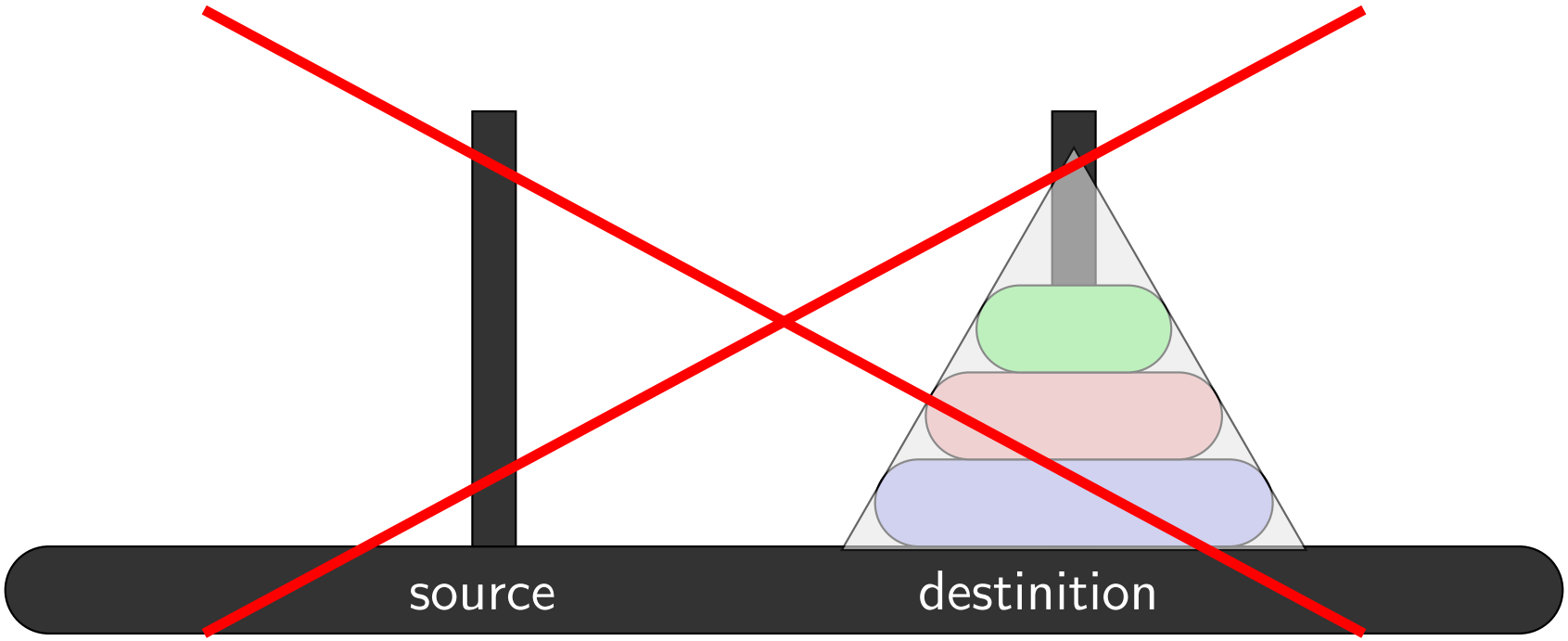
Tower of Hanoi



Tower of Hanoi



Tower of Hanoi



Rules

But it's not that simple.

Rules

But it's not that simple.

Suppose you have n disks

Rules

But it's not that simple.

Suppose you have n disks

- Only one disk can be moved among the towers at any given time.

Rules

But it's not that simple.

Suppose you have n disks

- Only one disk can be moved among the towers at any given time.
- Only the "top" disk can be removed.

Rules

But it's not that simple.

Suppose you have n disks

- Only one disk can be moved among the towers at any given time.
- Only the "top" disk can be removed.
- No large disk can sit over a small disk.

Rules

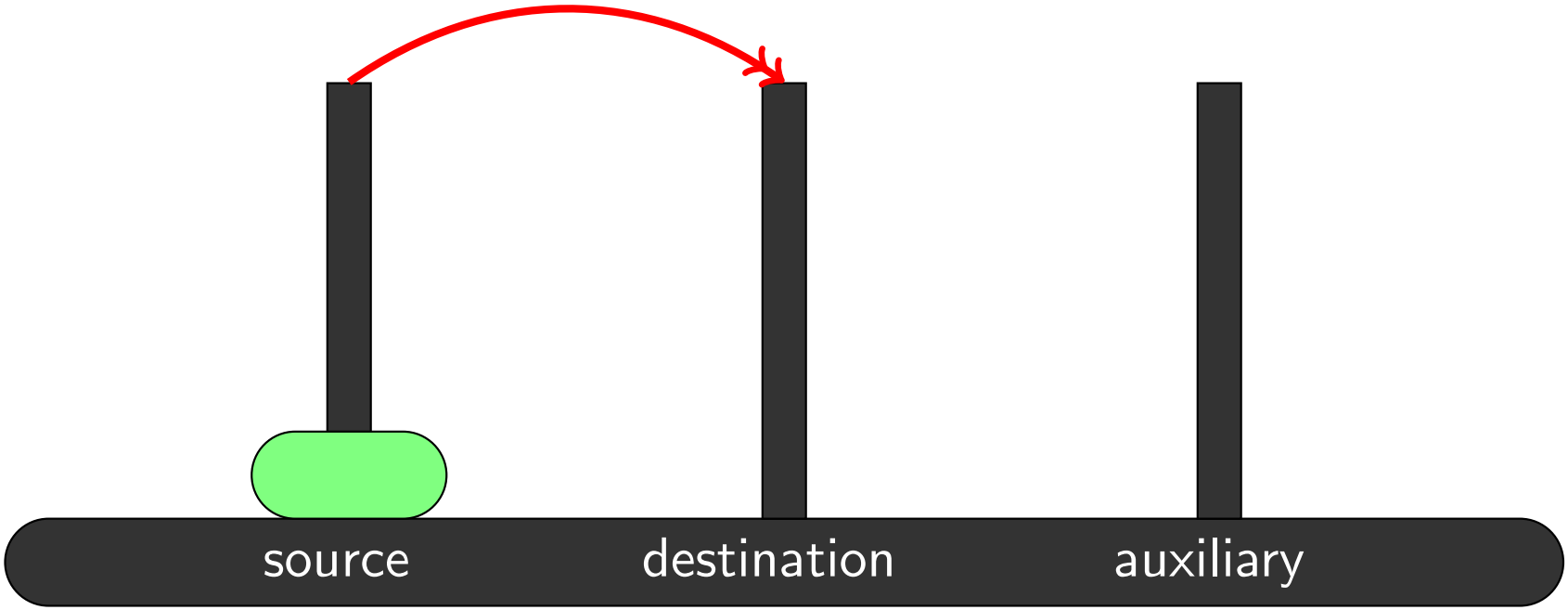
But it's not that simple.

Suppose you have n disks

- Only one disk can be moved among the towers at any given time.
- Only the "top" disk can be removed.
- No large disk can sit over a small disk.
- we can use an auxilary pole

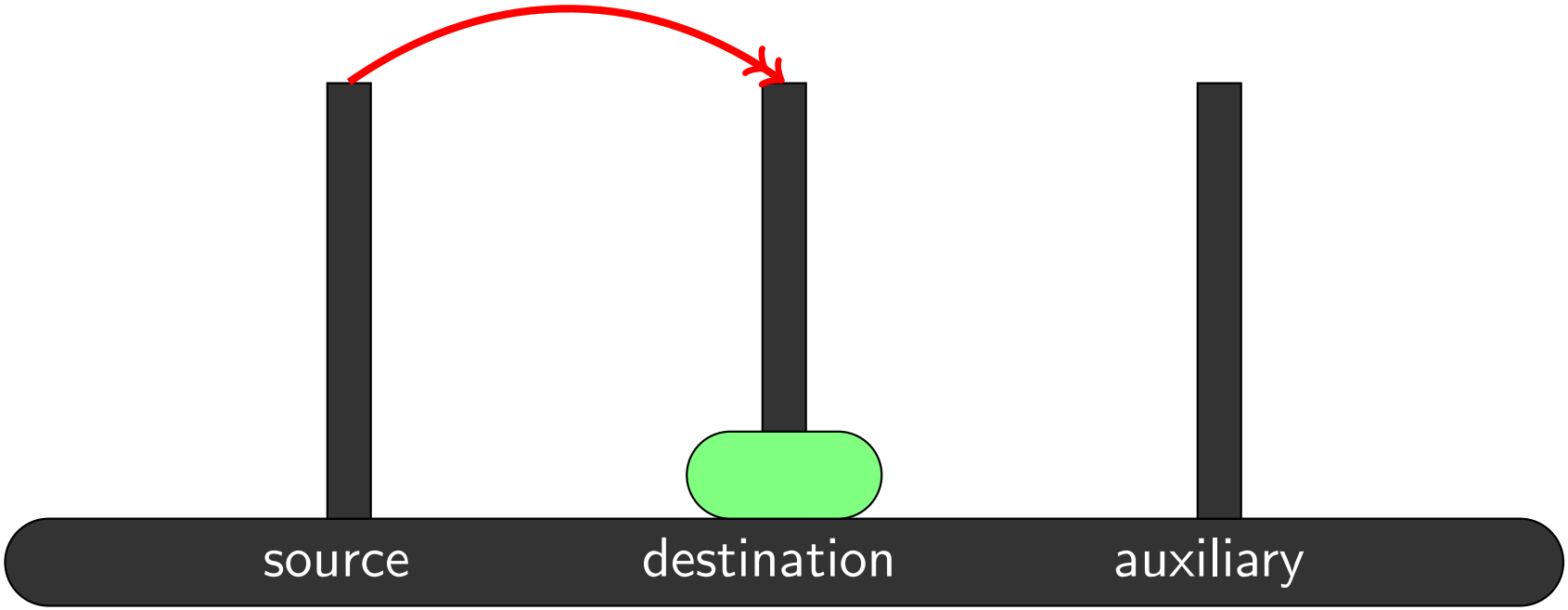
Tower of Hanoi

Consider n=1



Tower of Hanoi

Consider n=1



Algorithm

You need only one step

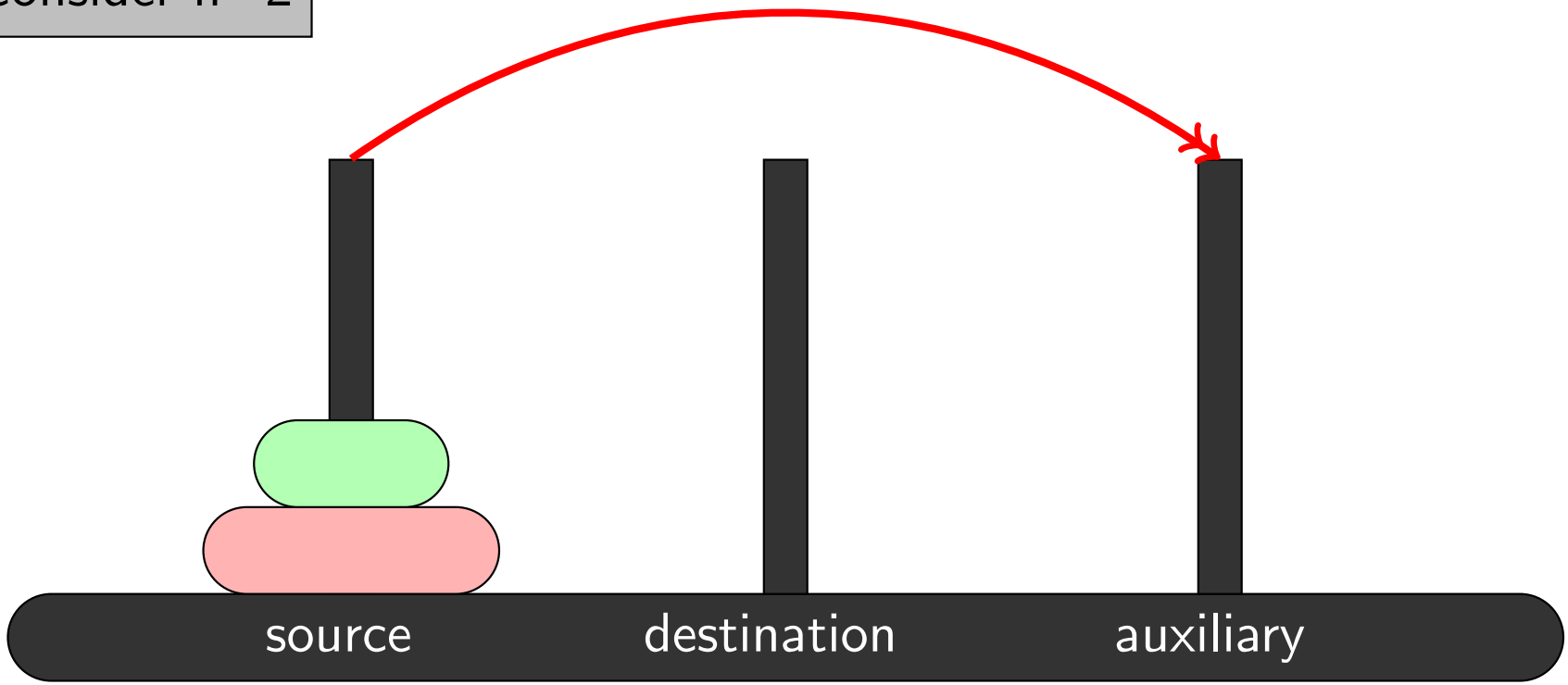
Algorithm

You need only one step

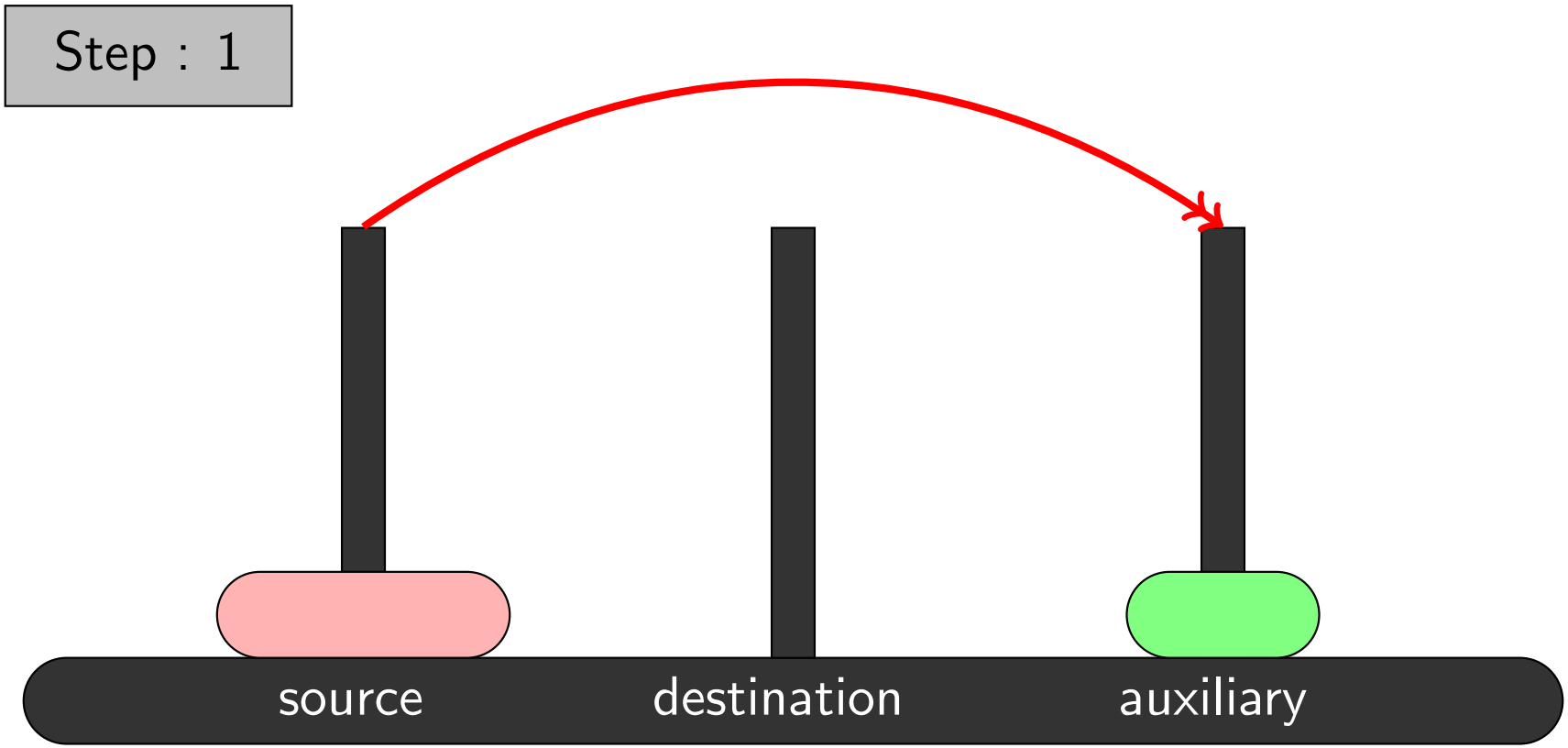
what if n=2 ??

Tower of Hanoi

Consider n=2

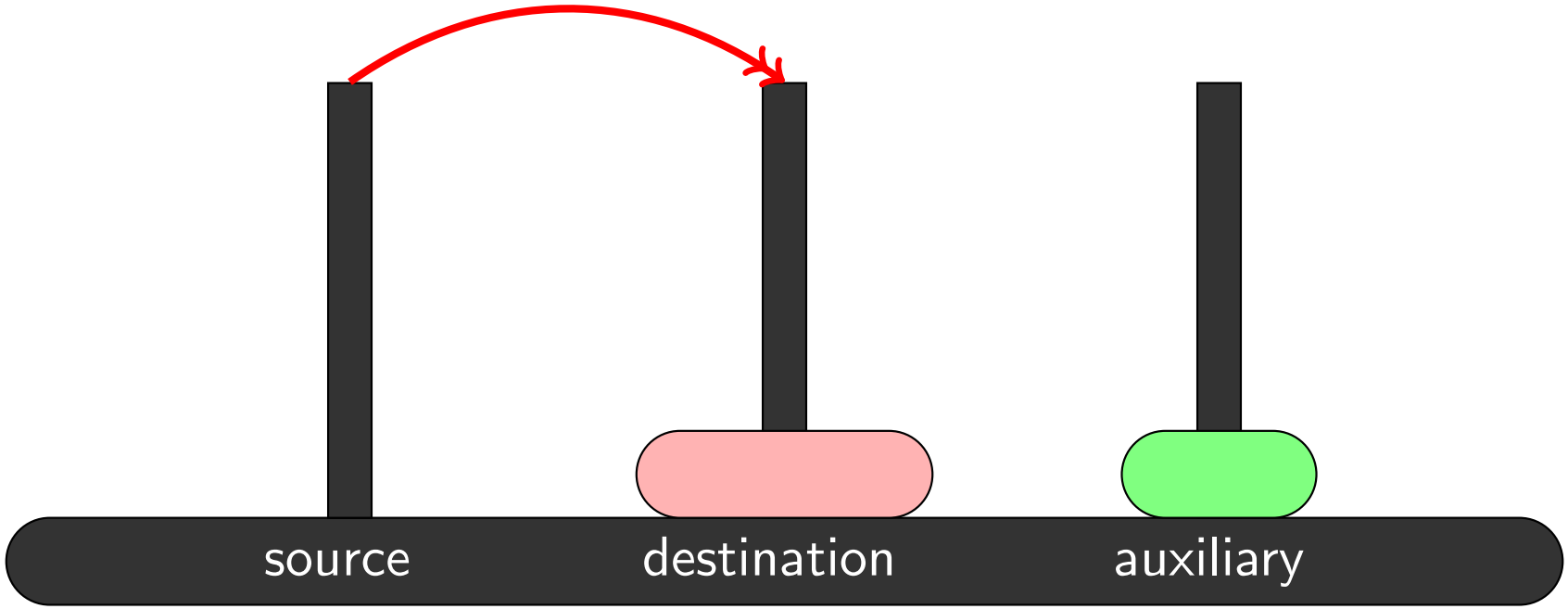


Tower of Hanoi



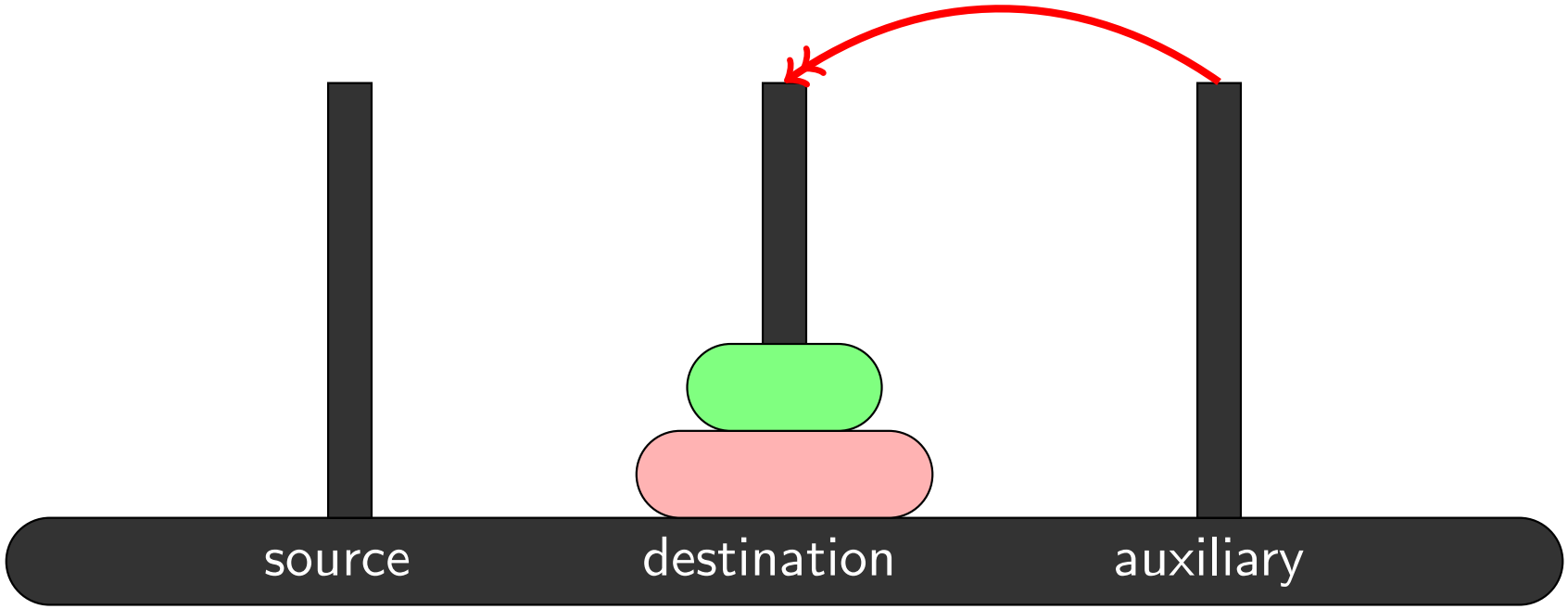
Tower of Hanoi

Step : 2



Tower of Hanoi

Step : 3



Algorithm

Can you find an algorithm ??

Algorithm

Can you find an algorithm ??

1. Move n-1 disks from source to aux

Algorithm

Can you find an algorithm ??

1. Move n-1 disks from source to aux
2. Move nth disk from source to dest

Algorithm

Can you find an algorithm ??

1. Move n-1 disks from source to aux
2. Move nth disk from source to dest
3. Move n-1 disks from aux to dest

Algorithm

Can you find an algorithm ??

1. Move n-1 disks from source to aux
2. Move nth disk from source to dest
3. Move n-1 disks from aux to dest

Still hard??

Pseudocode

```
Hanoi(disk - 1, source, aux, dest)      // Step 1
move disk from source to dest)          // Step 2
Hanoi(disk - 1, aux, dest, source)      // Step 3
```

Pseudocode

```
START
    Procedure Hanoi(disk, source, dest, aux)
        IF disk == 1, THEN
            move disk from source to dest
        ELSE
            Hanoi(disk - 1, source, aux, dest)      // Step 1
            move disk from source to dest)          // Step 2
            Hanoi(disk - 1, aux, dest, source)      // Step 3
        END IF
    END Procedure
STOP
```

Recursion Tree

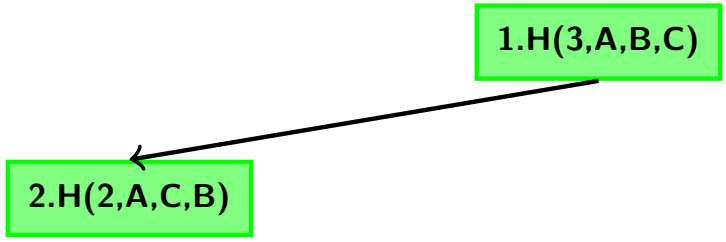
Now, lets see the corresponding Recursion tree for 3 Disk.

1. A is the source pole
2. B is the destination pole
3. C is the auxiliary pole

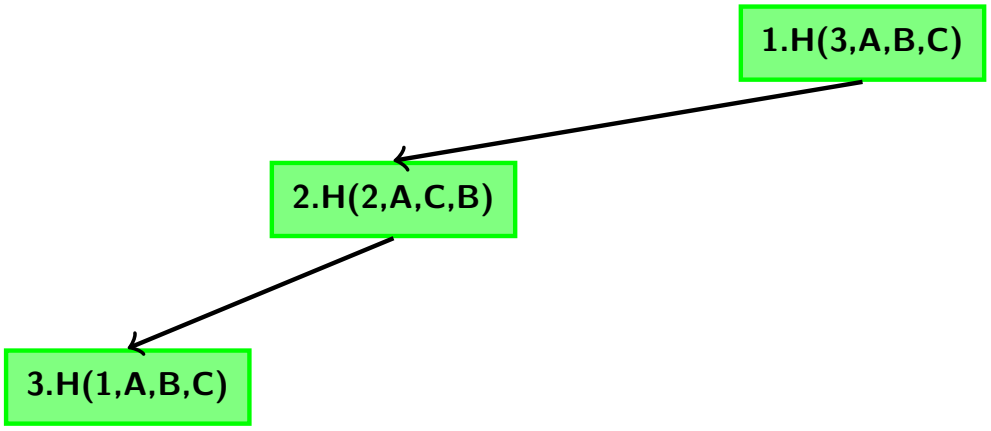
Recursion Tree (For disk = 3)

1.H(3,A,B,C)

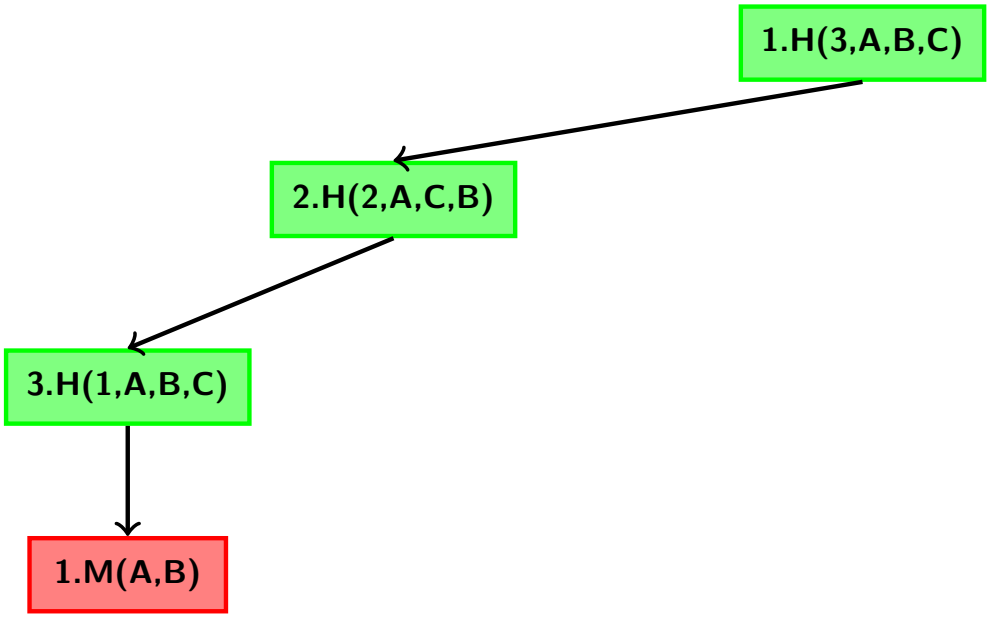
Recursion Tree (For disk = 3)



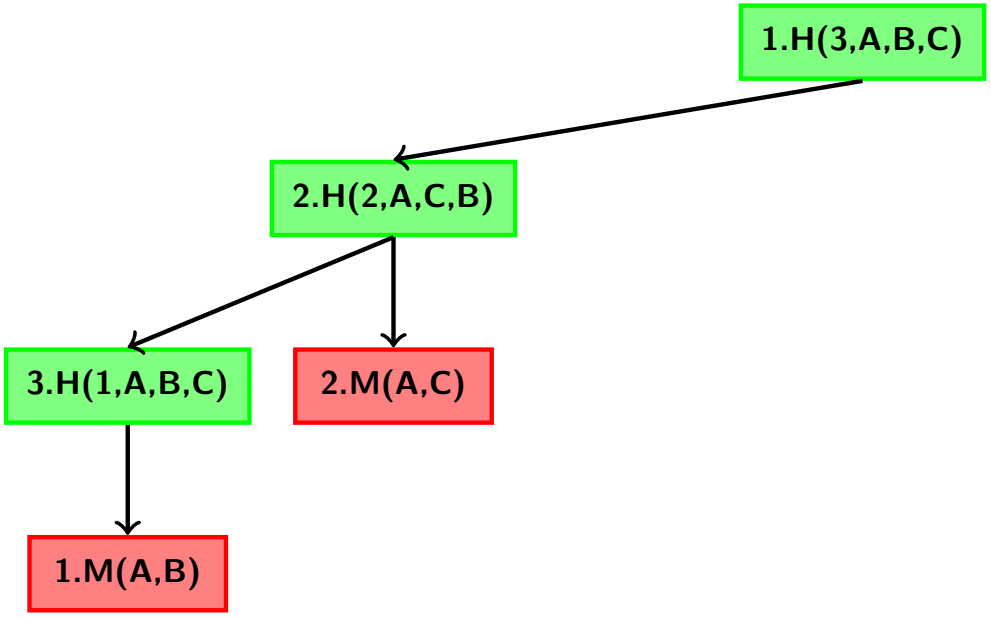
Recursion Tree (For disk = 3)



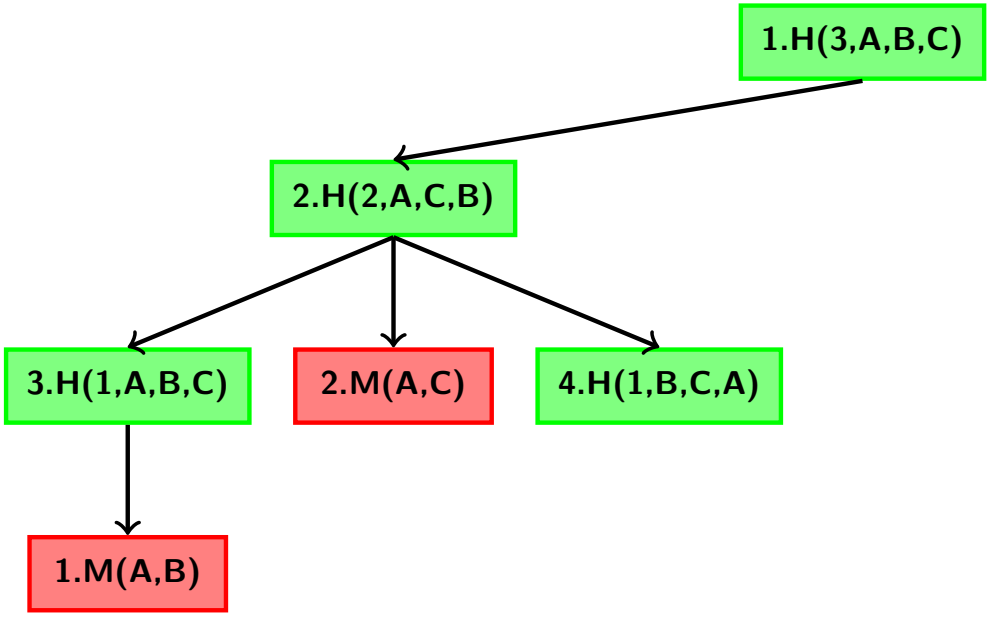
Recursion Tree (For disk = 3)



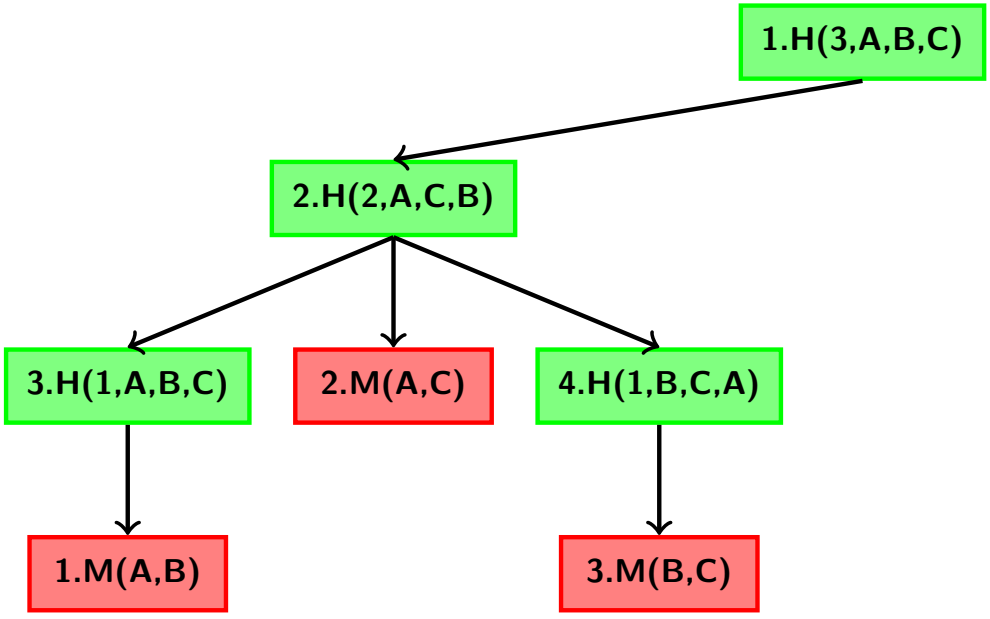
Recursion Tree (For disk = 3)



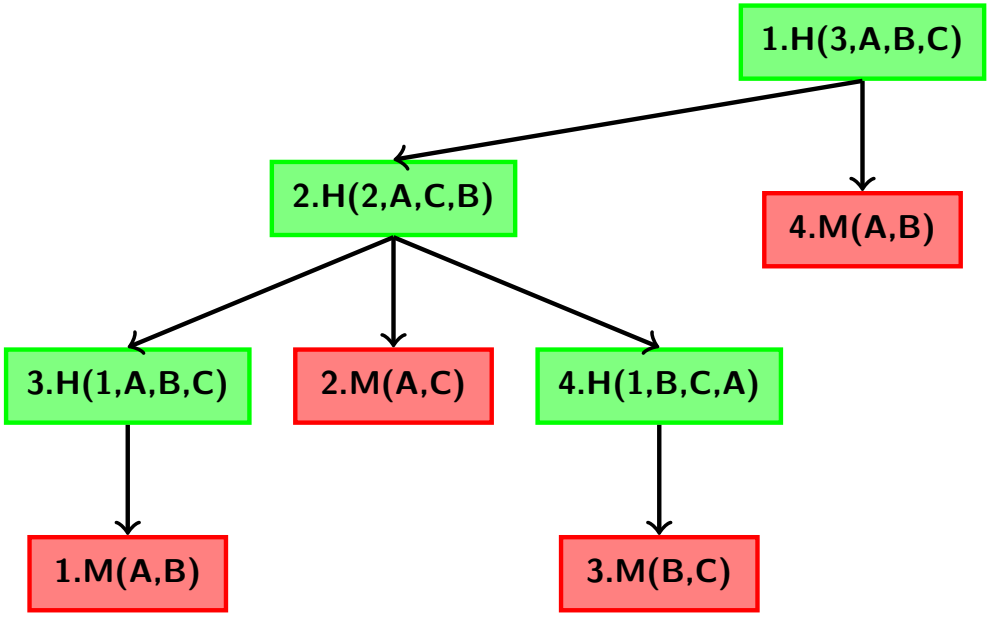
Recursion Tree (For disk = 3)



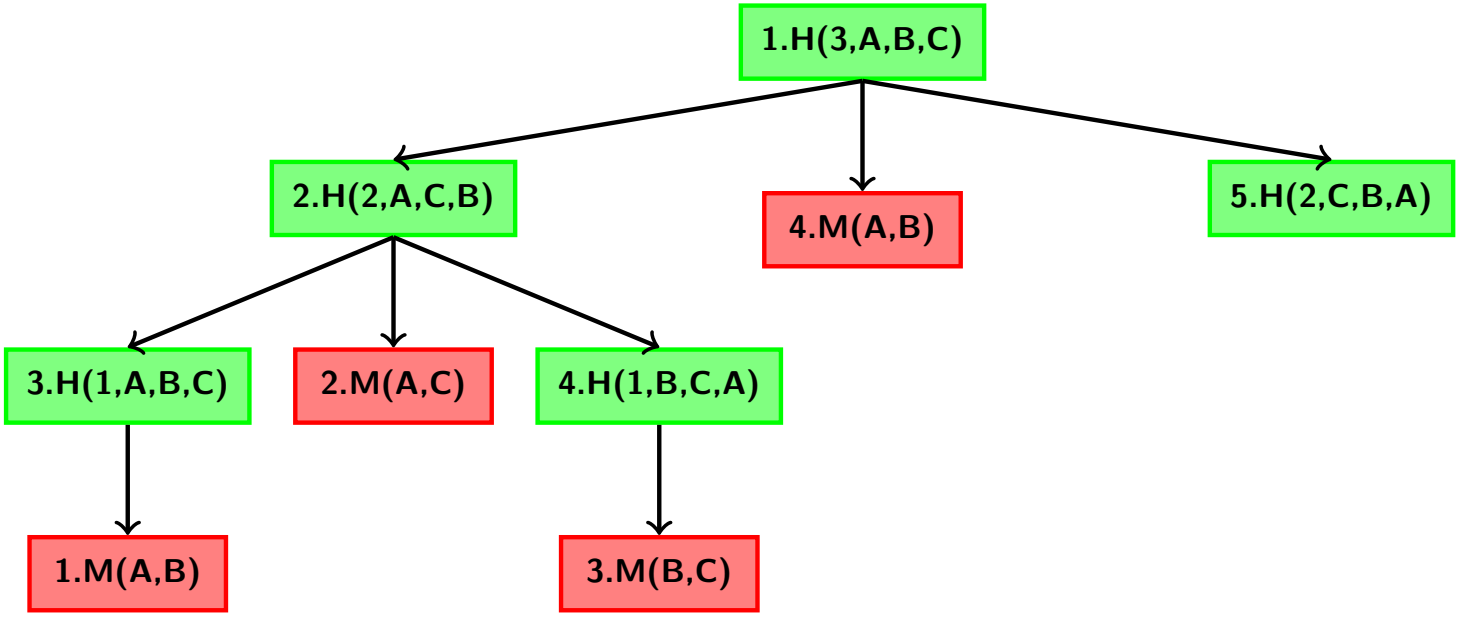
Recursion Tree (For disk = 3)



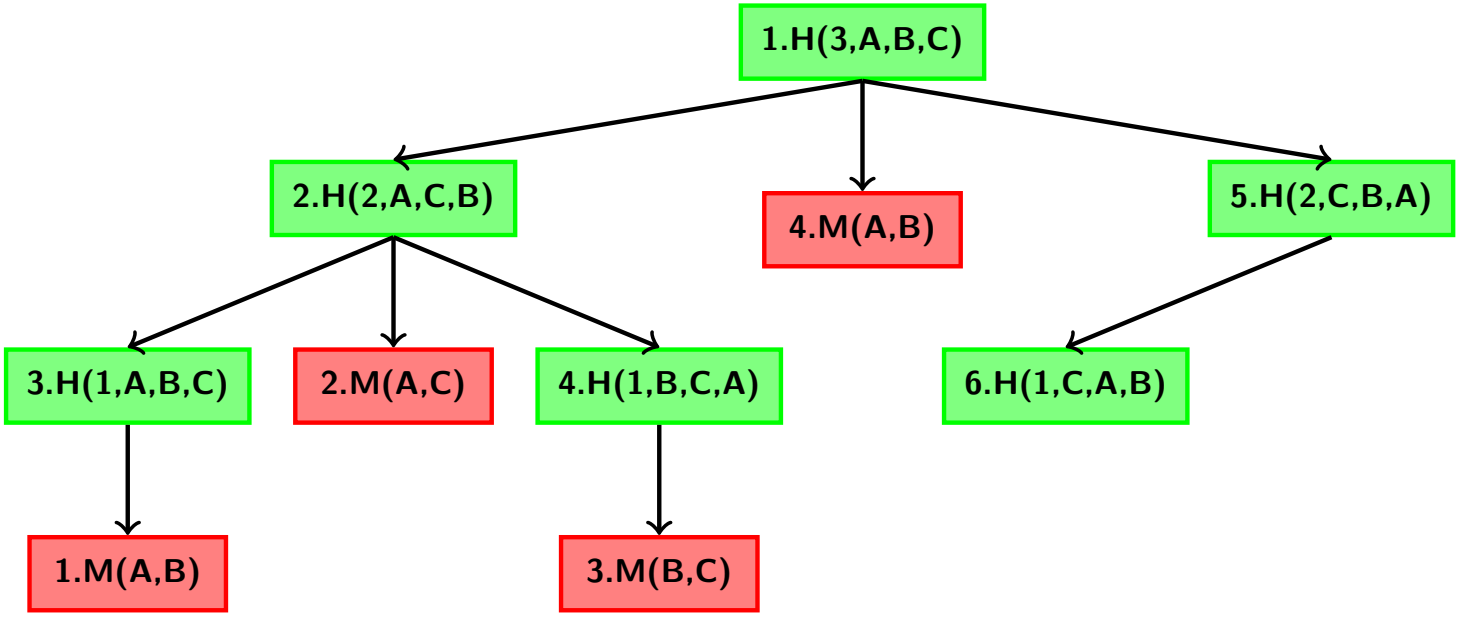
Recursion Tree (For disk = 3)



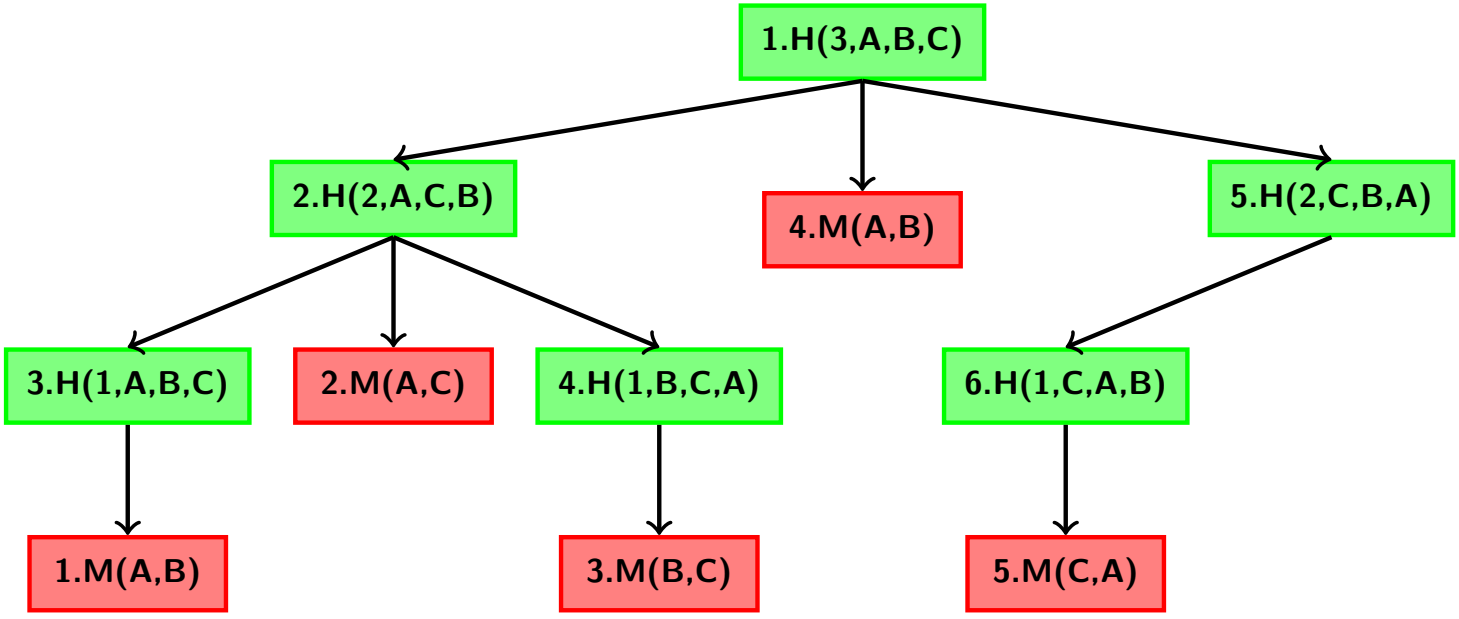
Recursion Tree (For disk = 3)



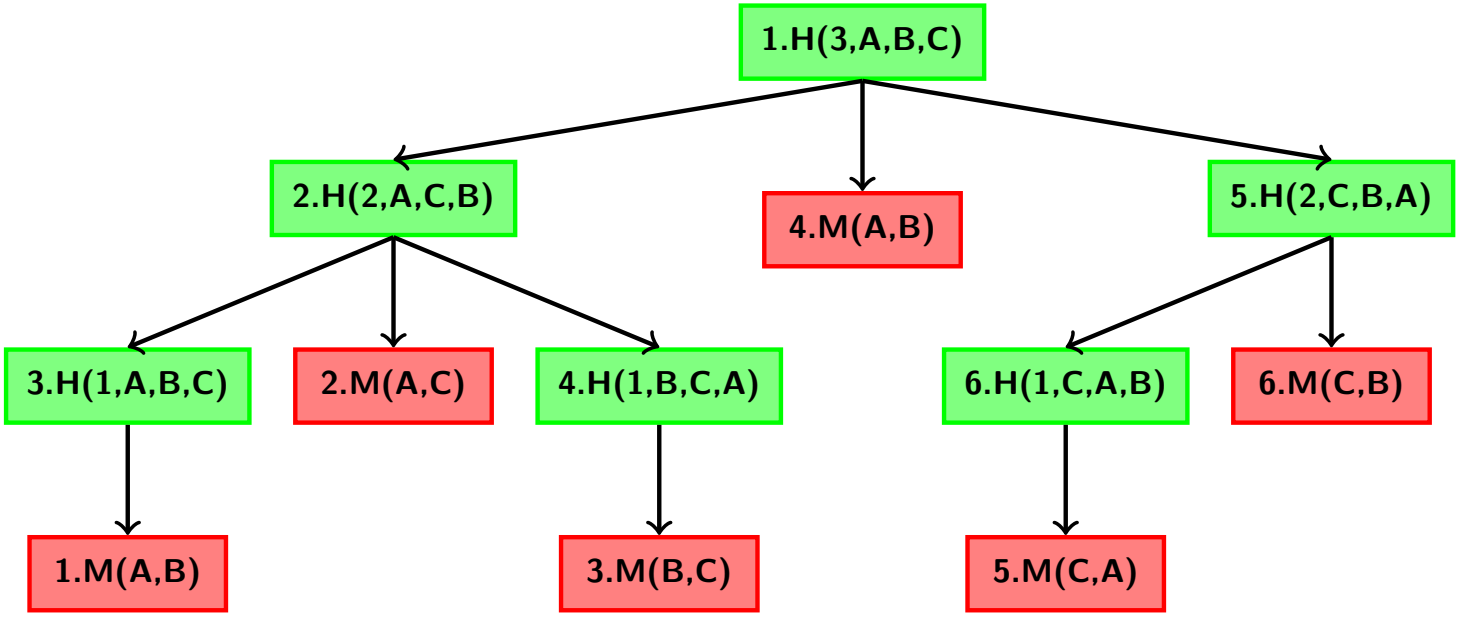
Recursion Tree (For disk = 3)



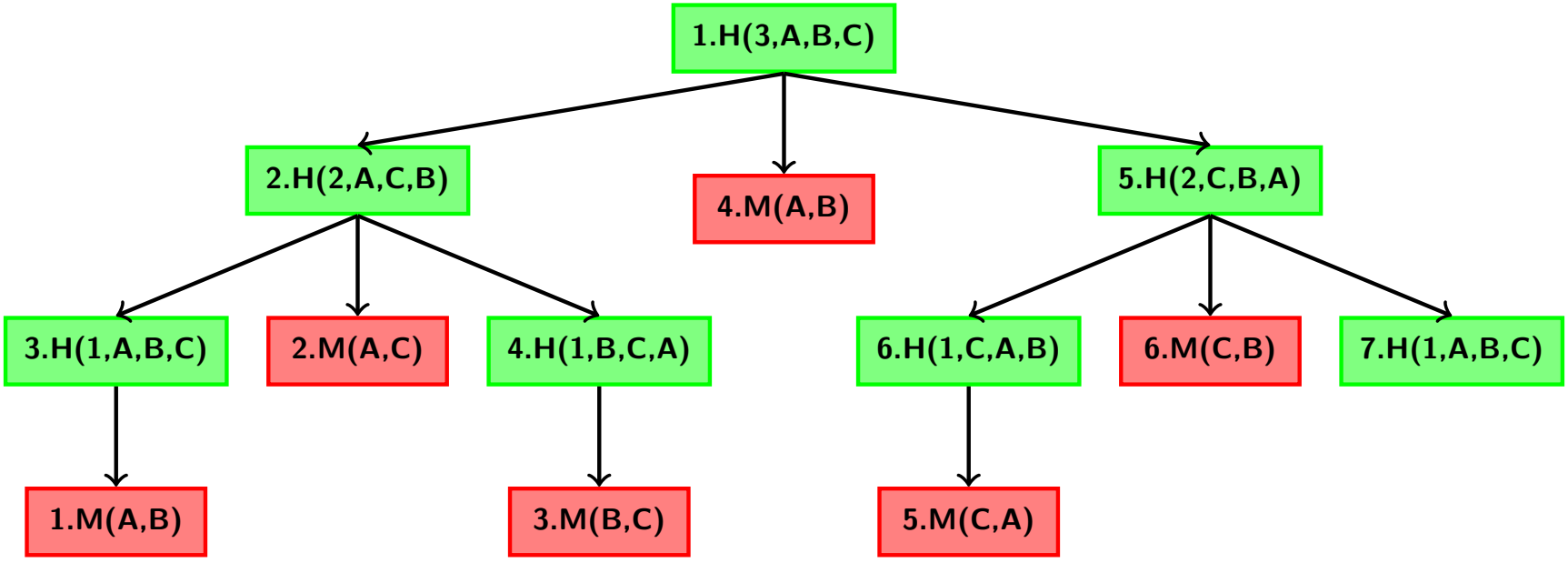
Recursion Tree (For disk = 3)



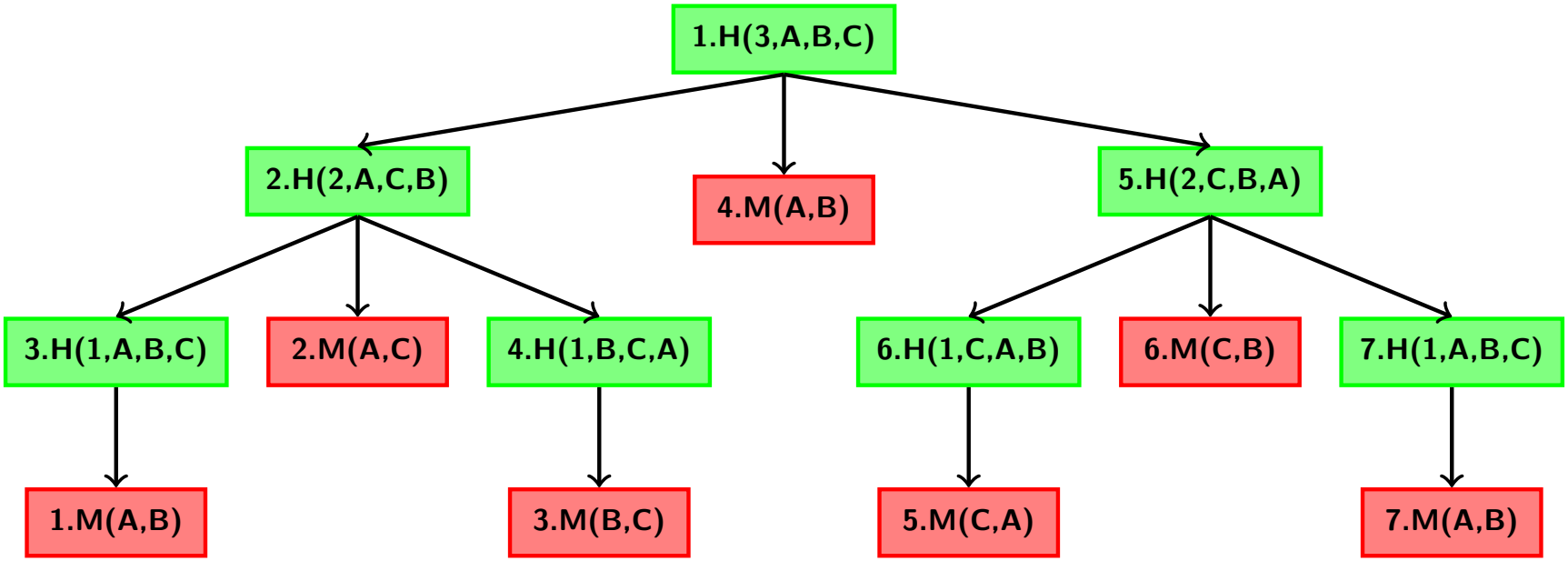
Recursion Tree (For disk = 3)



Recursion Tree (For disk = 3)



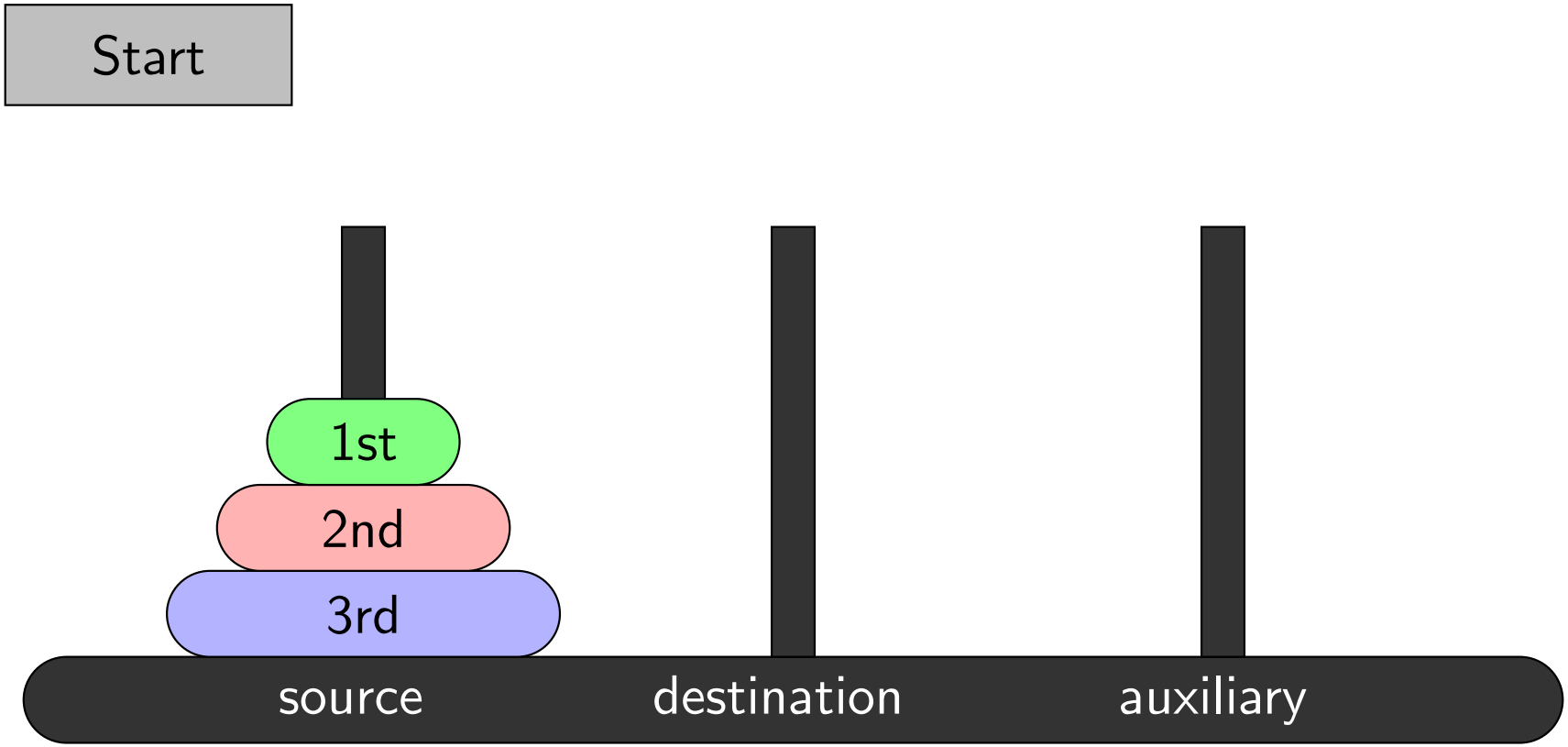
Recursion Tree (For disk = 3)



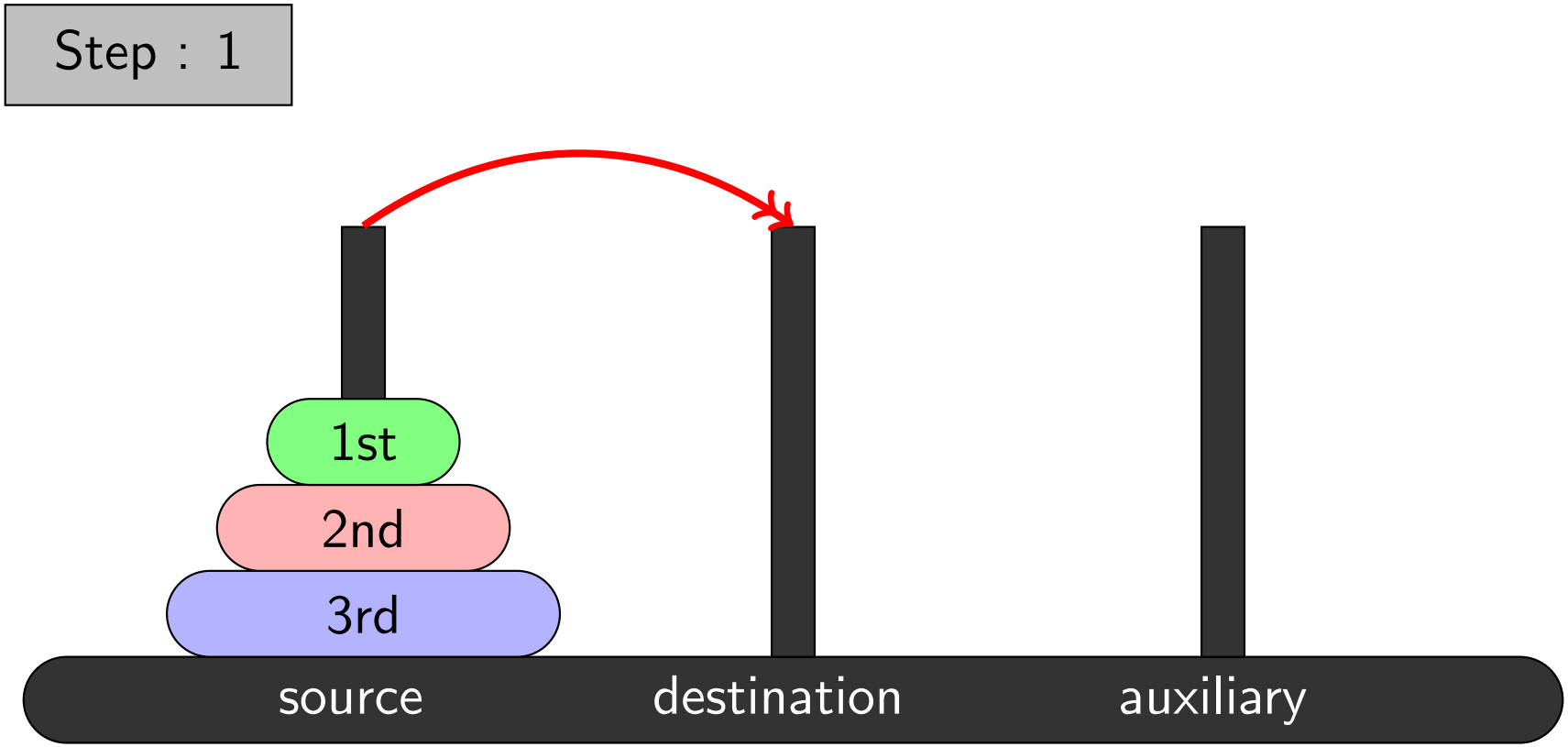
Tower of Hanoi

Lets see the simulation for the leaf nodes of the tree.
And try to visualize what's happening.

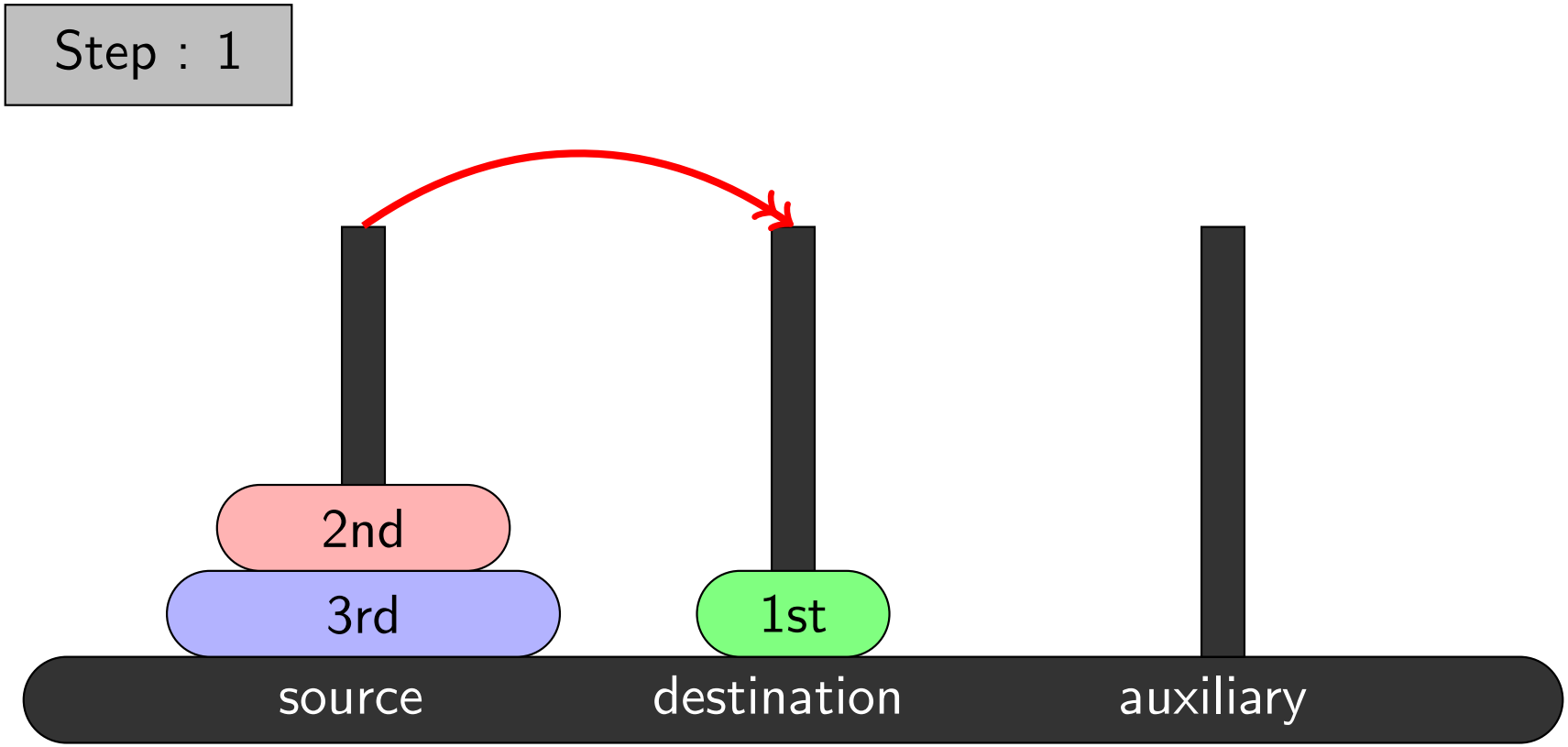
Tower of Hanoi



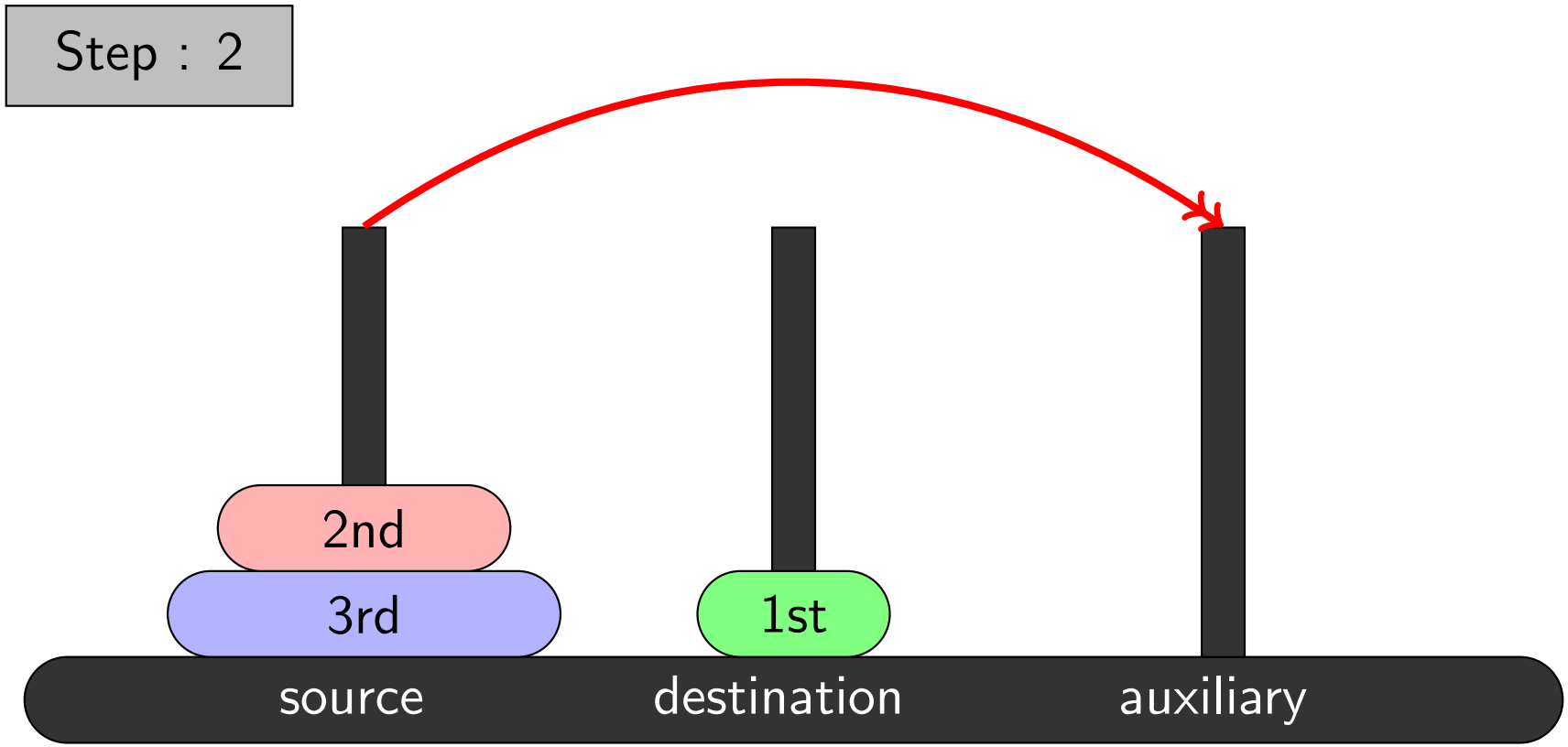
Tower of Hanoi



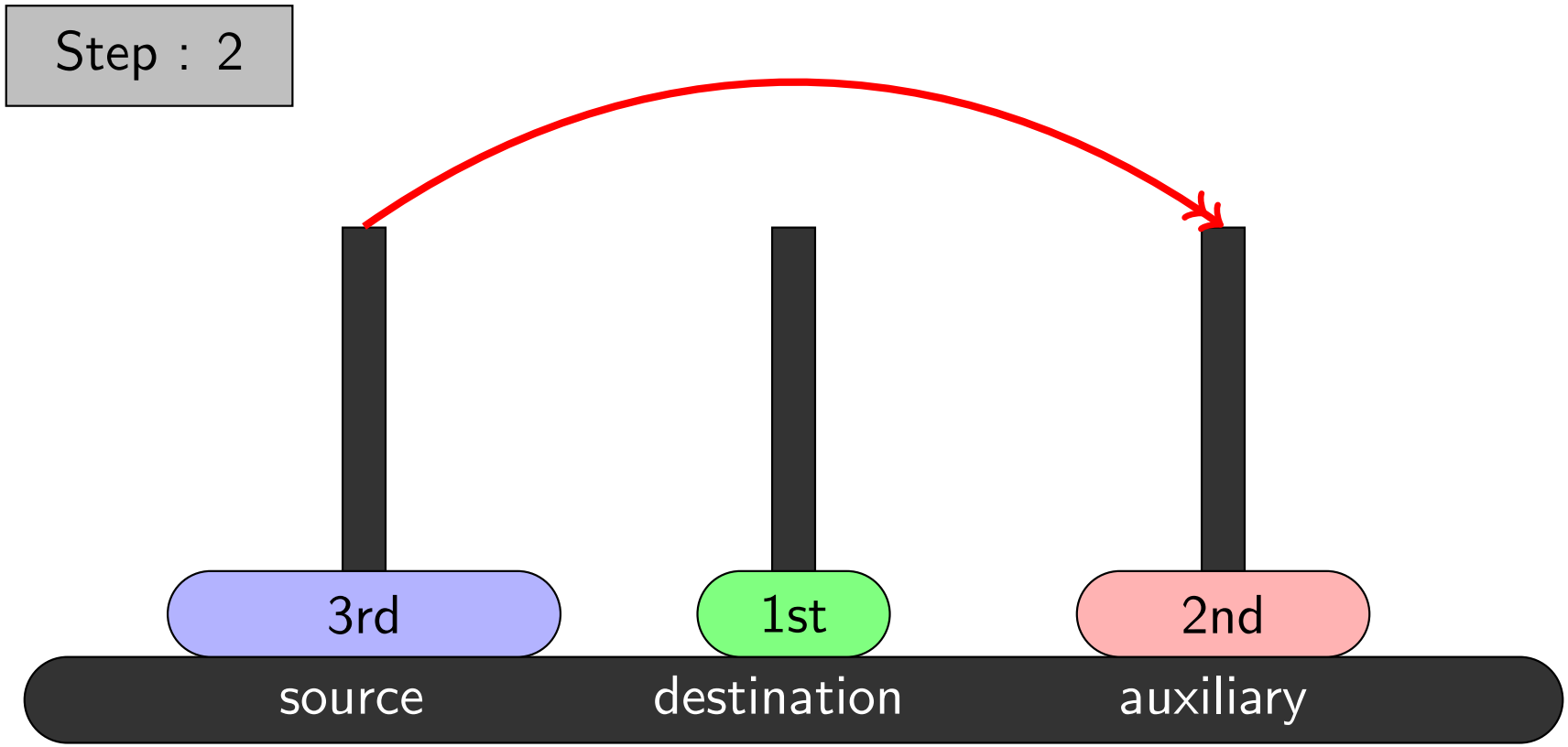
Tower of Hanoi



Tower of Hanoi

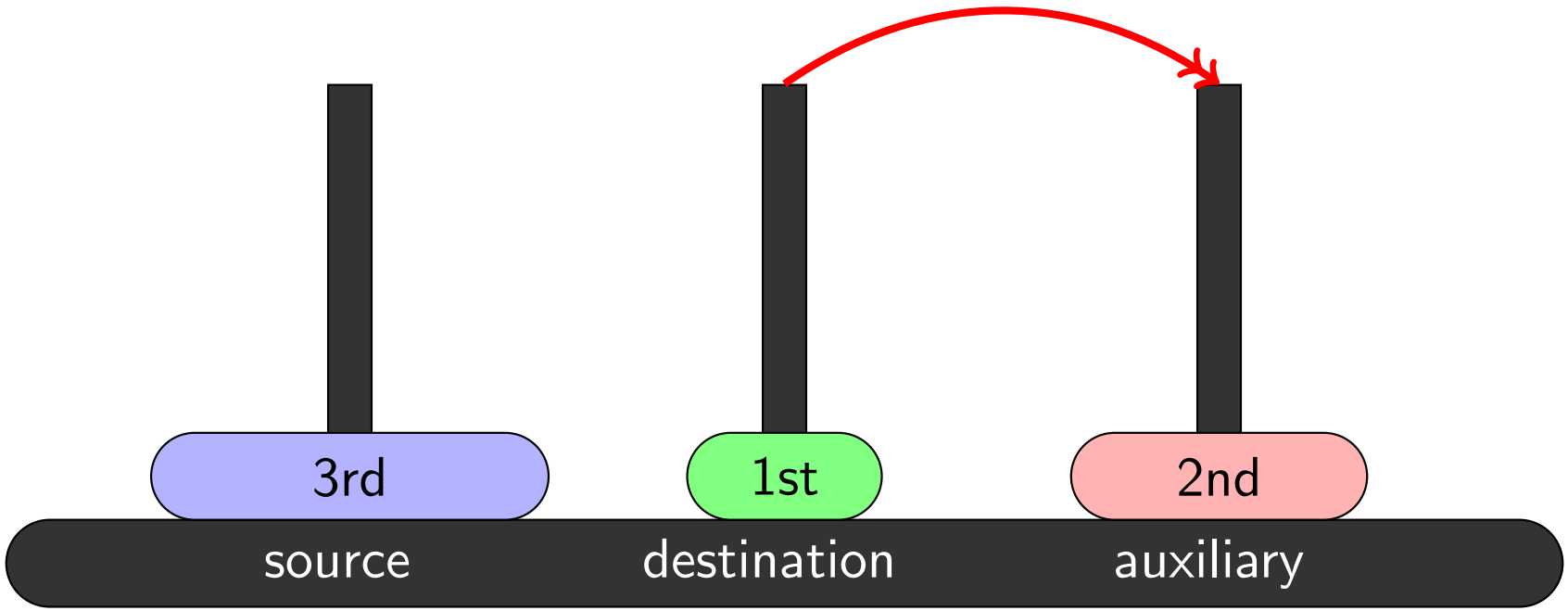


Tower of Hanoi



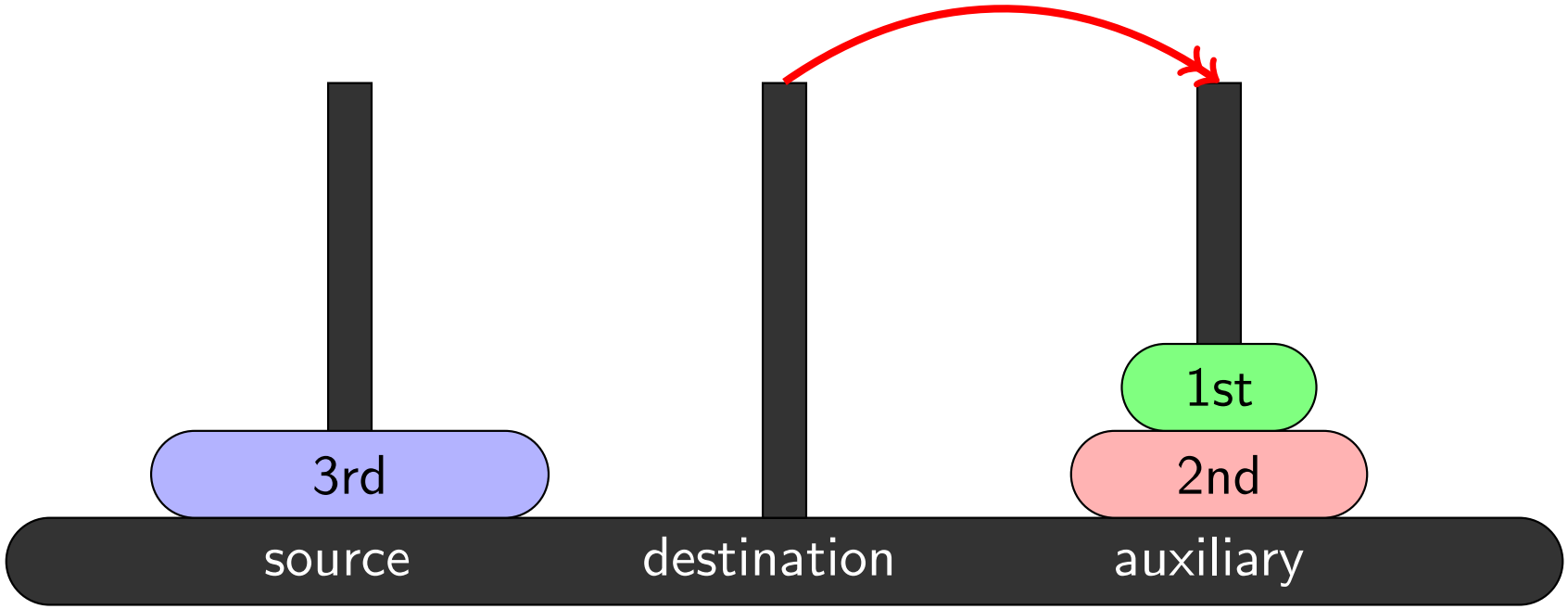
Tower of Hanoi

Step : 3



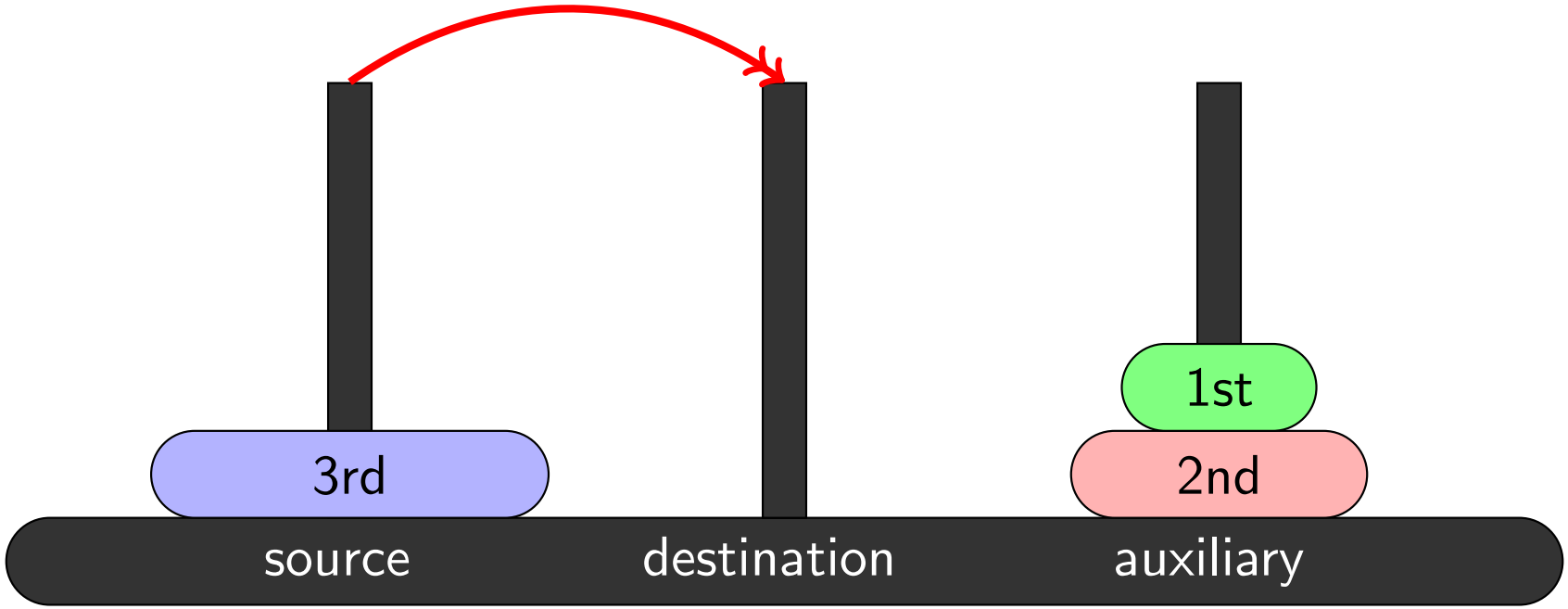
Tower of Hanoi

Step : 3



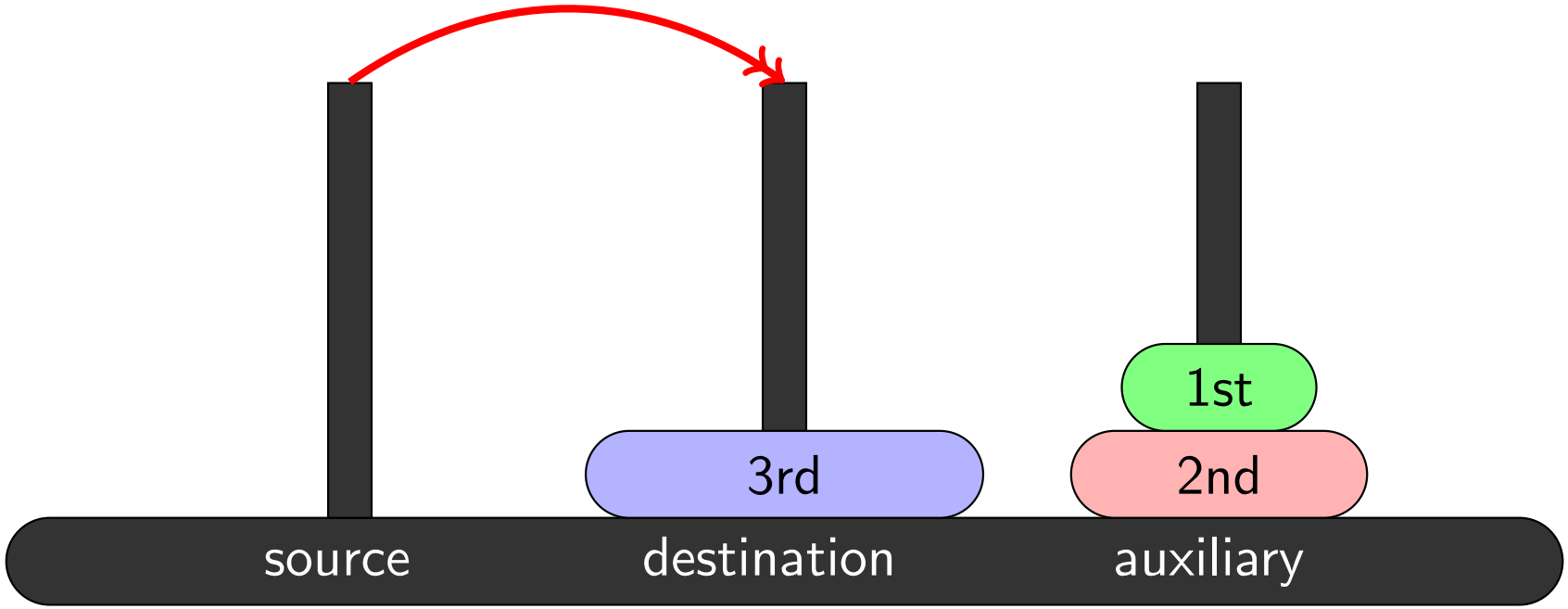
Tower of Hanoi

Step : 4

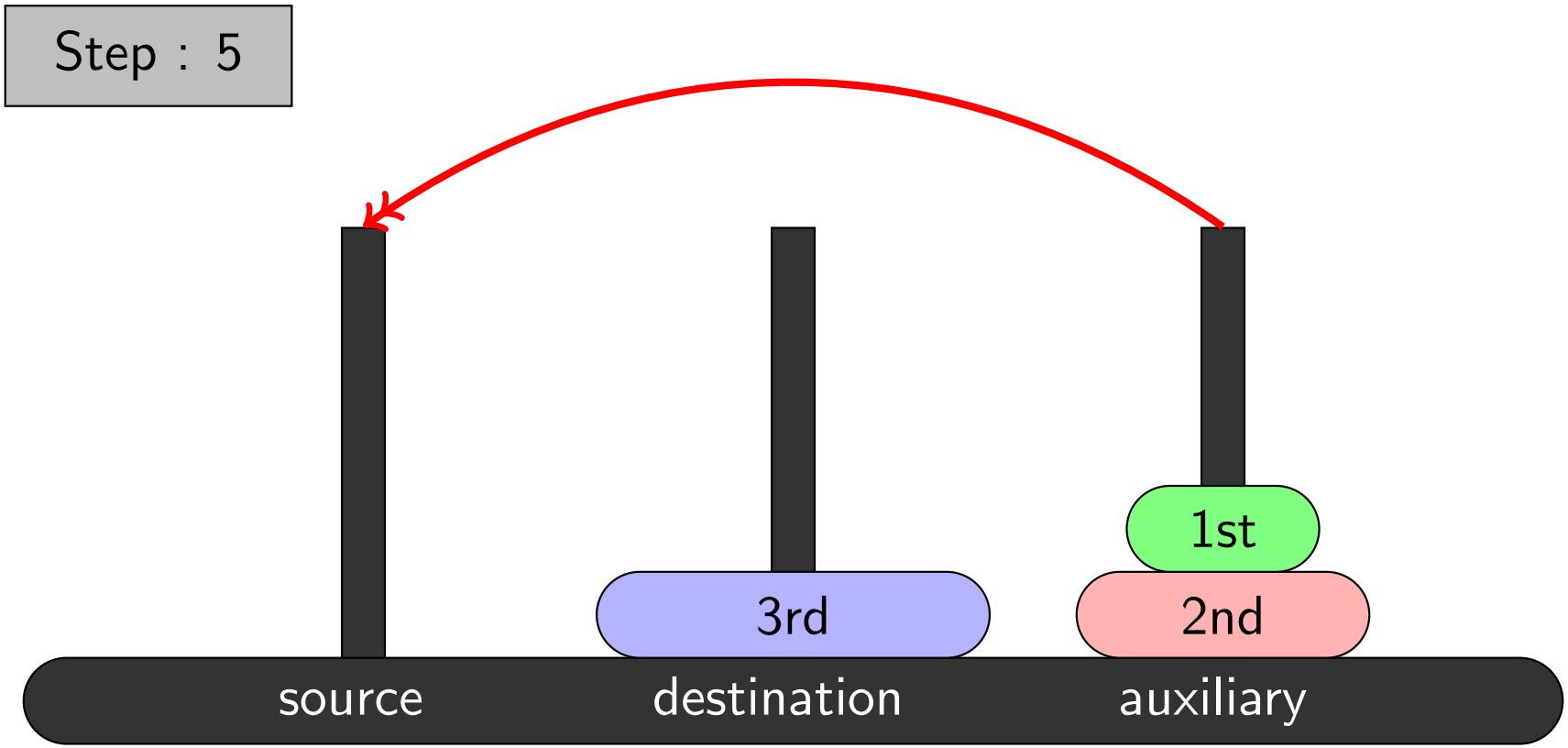


Tower of Hanoi

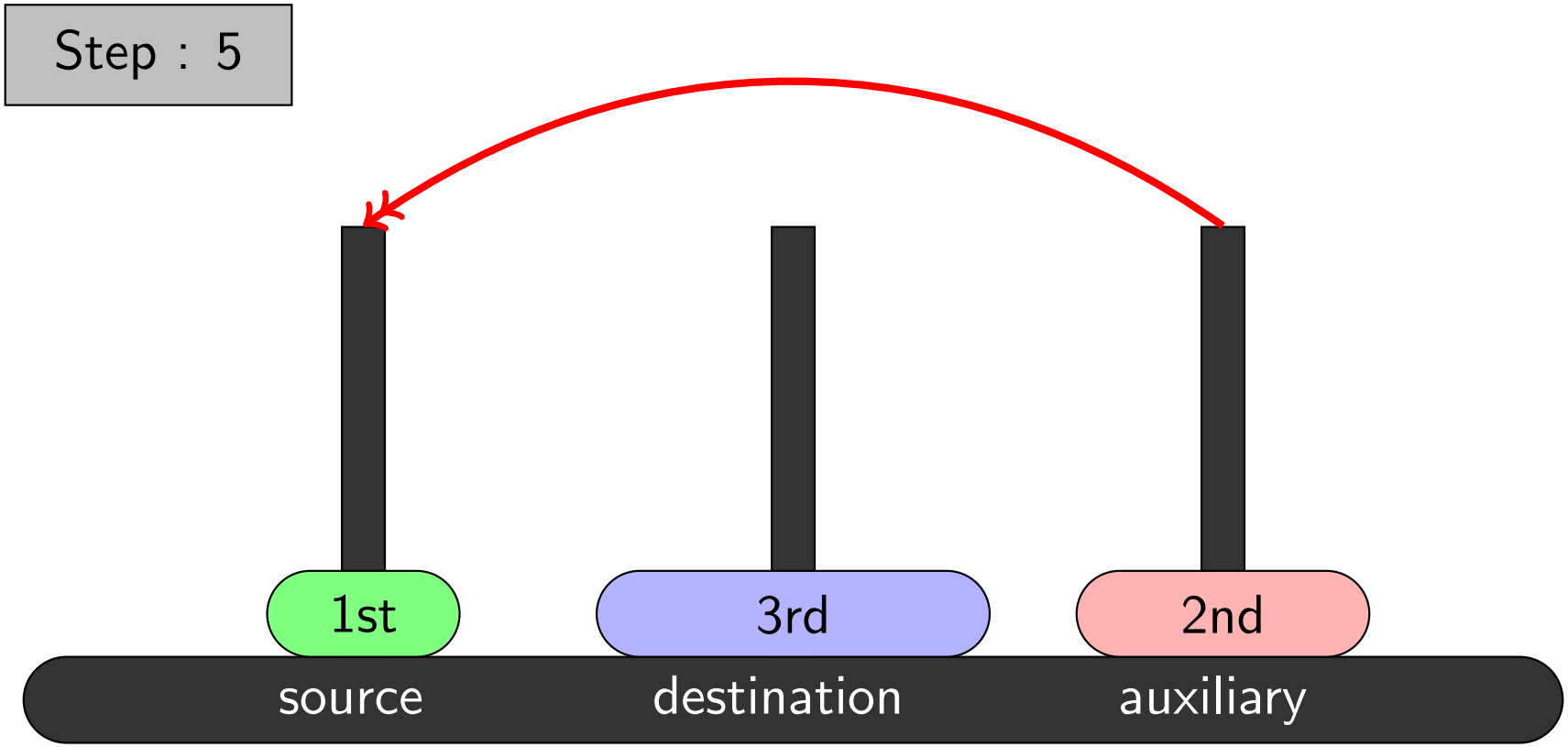
Step : 4



Tower of Hanoi

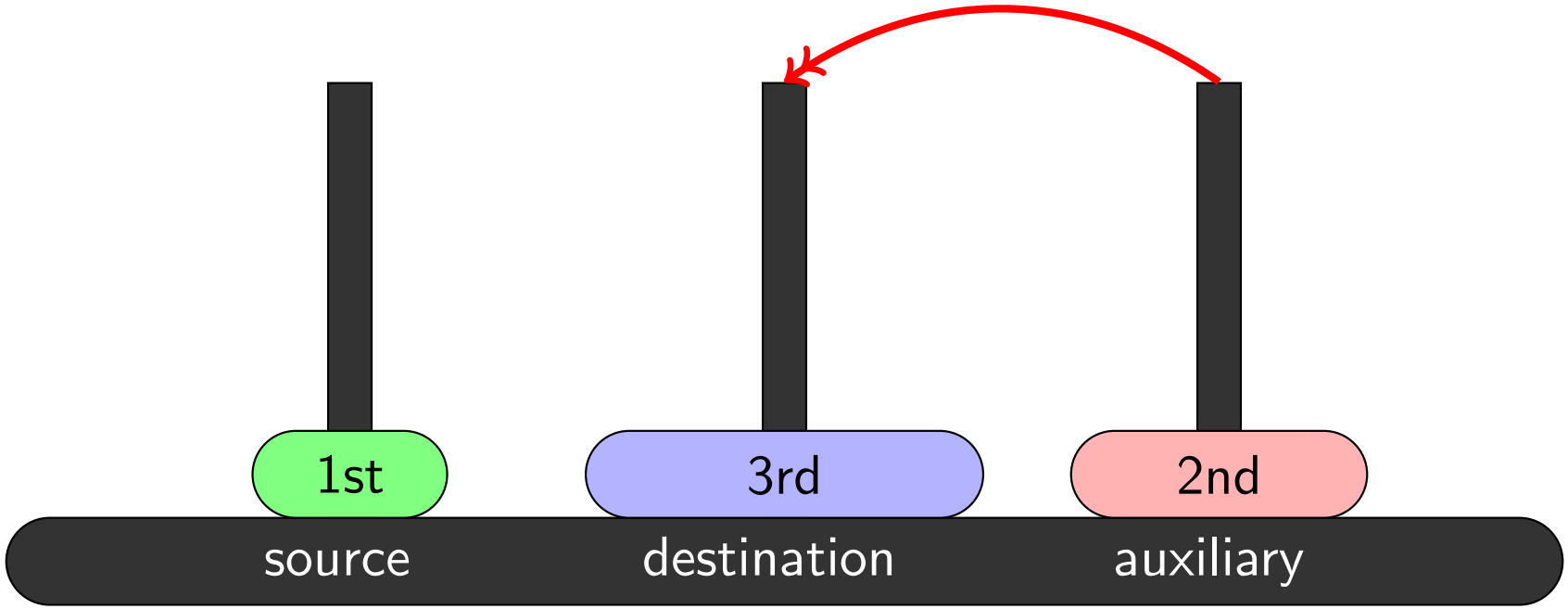


Tower of Hanoi



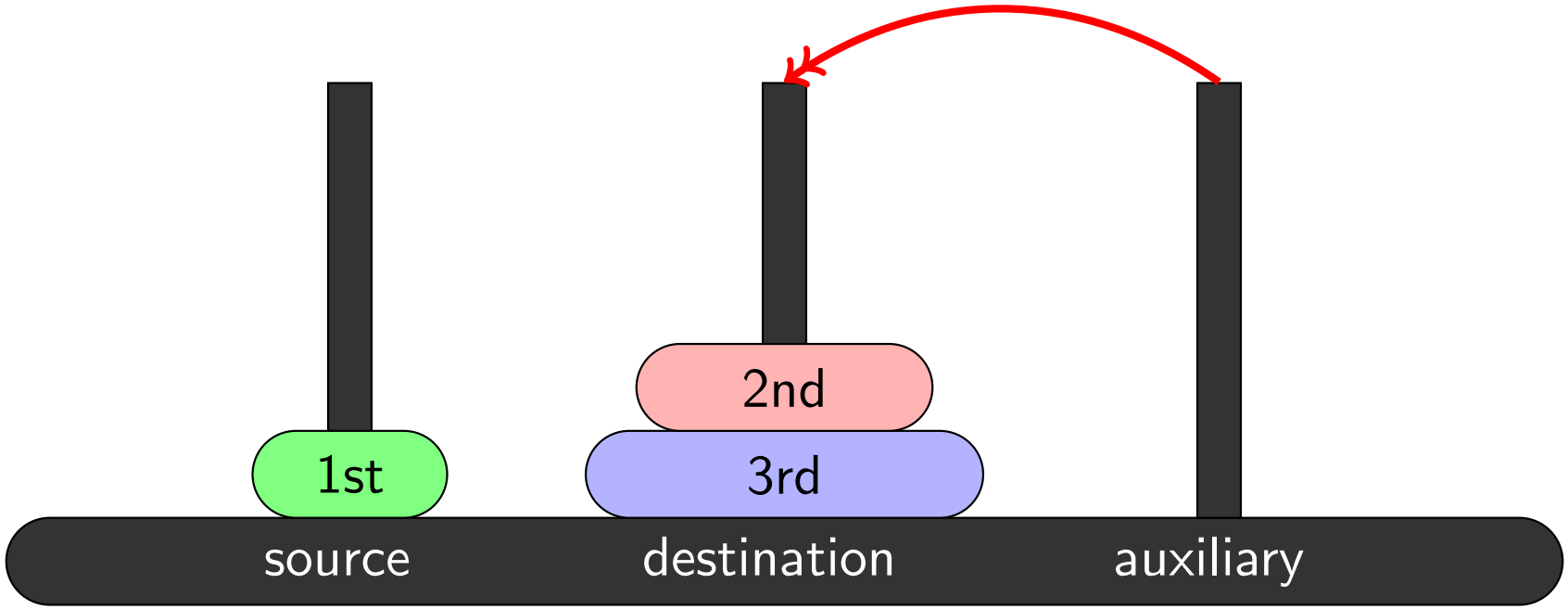
Tower of Hanoi

Step : 6

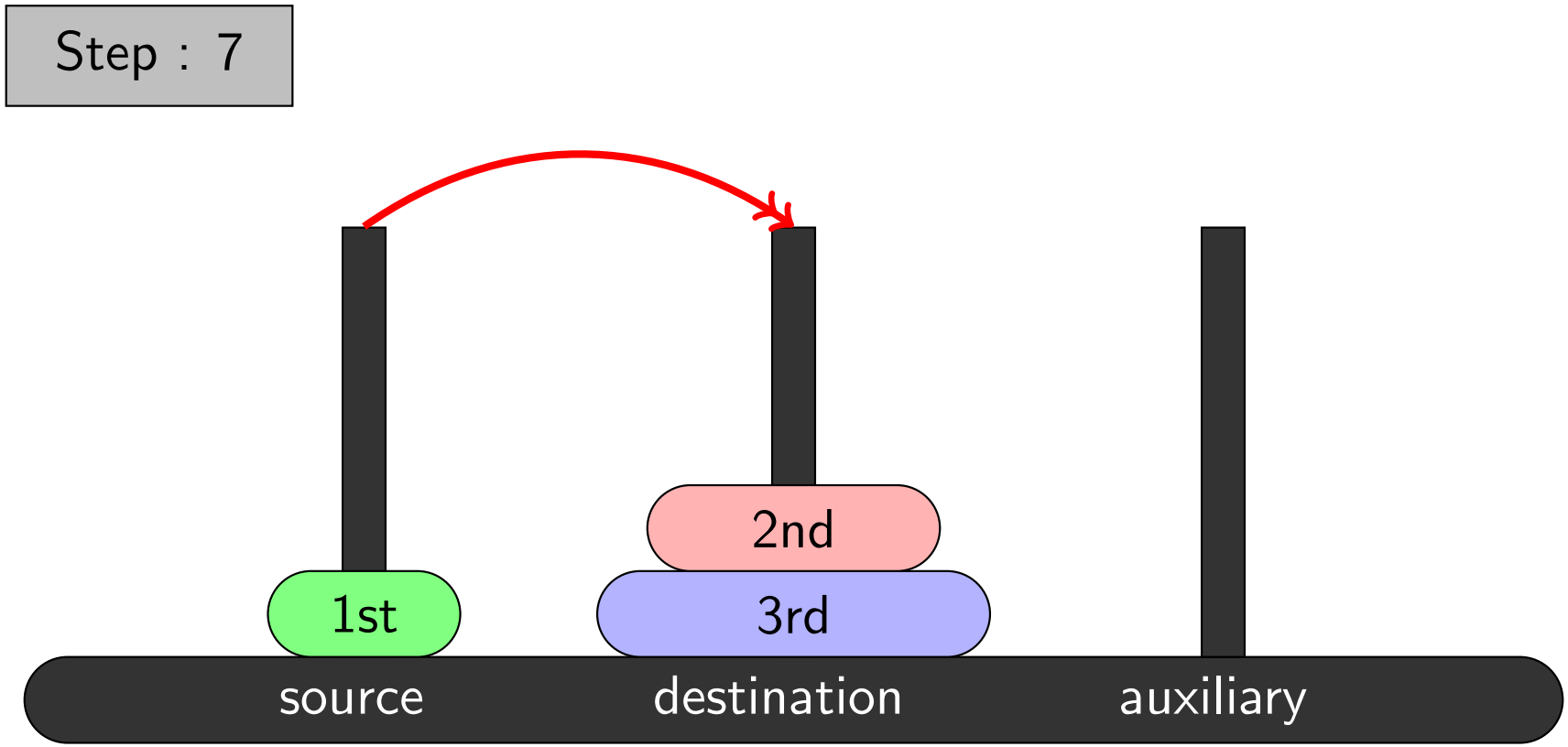


Tower of Hanoi

Step : 6

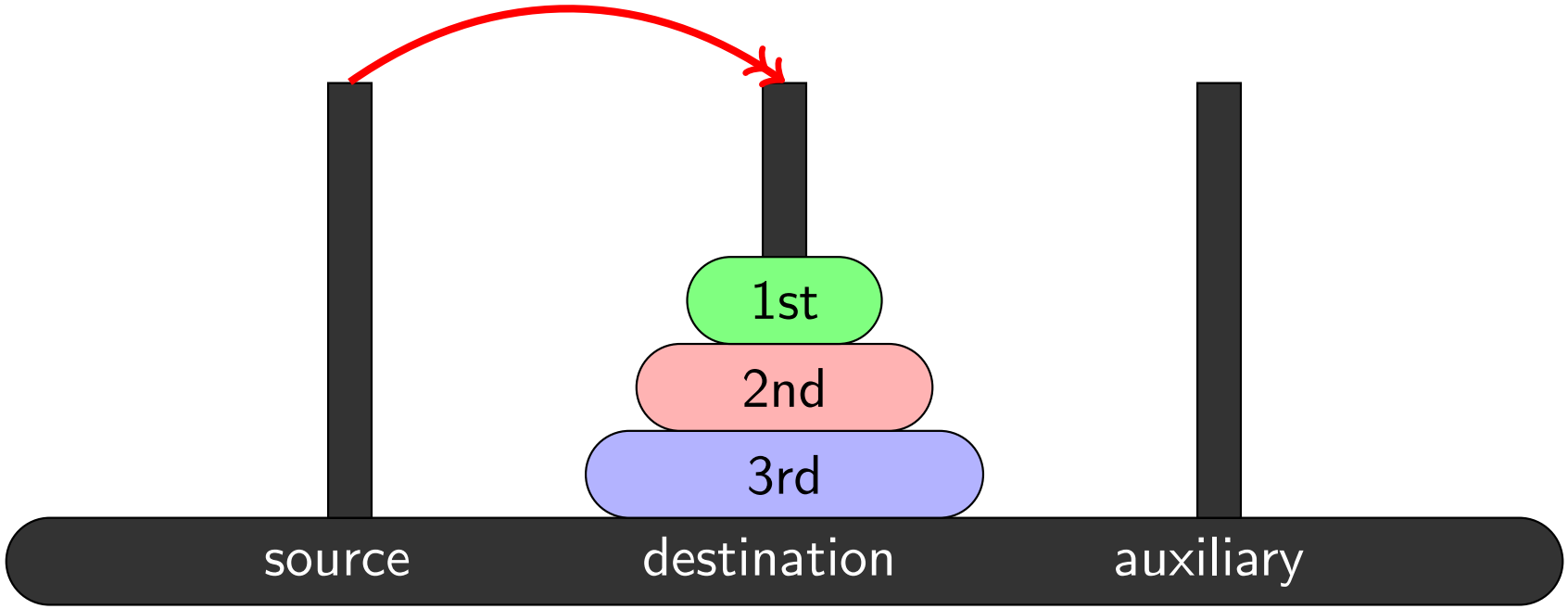


Tower of Hanoi



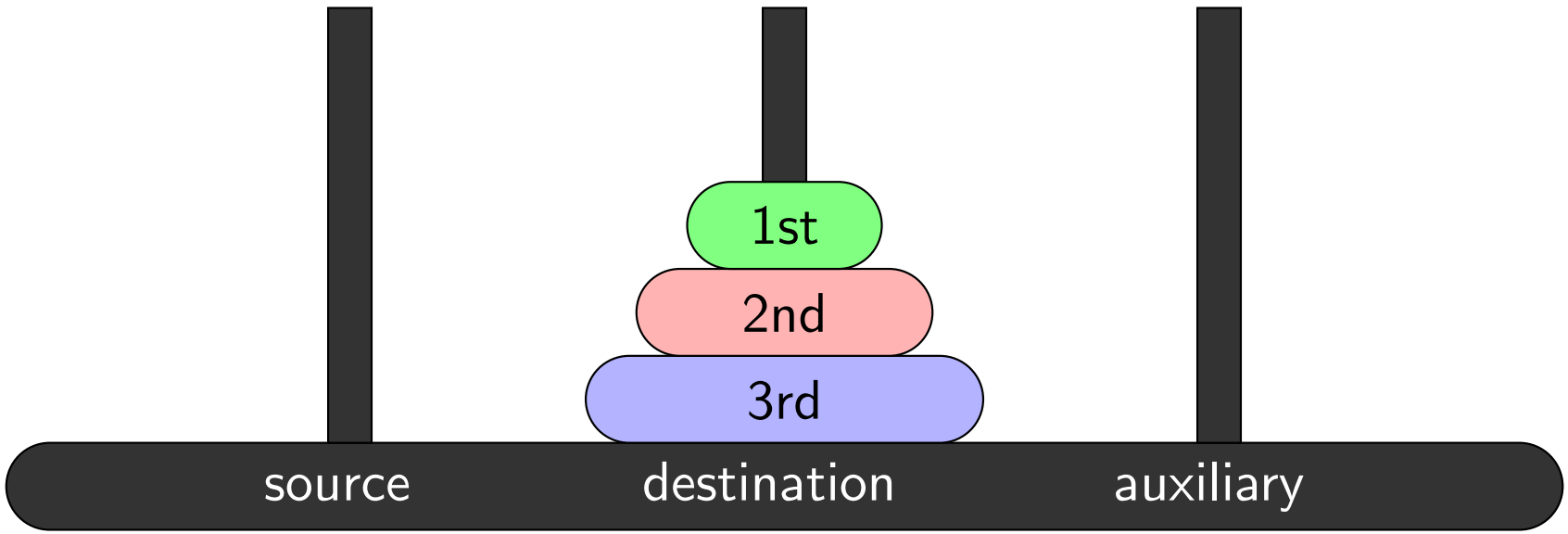
Tower of Hanoi

Step : 7



Tower of Hanoi

Done



Time Complexity

- What will be the time complexity to solve the problem.

Time Complexity

Recursive Equation :

$$T(n) = 2T(n-1) + 1 \tag{1}$$

Solving it by Backsubstitution :

$$T(n-1) = 2T(n-2) + 1 \tag{2}$$

$$T(n-2) = 2T(n-3) + 1 \tag{3}$$

Time Complexity

Put the value of $T(n-2)$ in eq-2 from eq-3...

$$T(n-1) = 2(2T(n-3)+1)+1 \tag{4}$$

Put the value of $T(n-1)$ in eq-1 from eq-4...

$$T(n) = 2(2(2T(n-3)+1)+1)+1$$

$$=> T(n) = 2^3 T(n-3) + 2^2 + 2^1 + 1$$

After Generalization :

$$T(n) = 2^k T(n-k) + 2^{(k-1)} + 2^{(k-2)} + ... + 2^2 + 2^1 + 1 \tag{5}$$

Time Complexity

Base condition :

$$T(1) = 1 \tag{6}$$

Relation between n and k from eq-6...

$$(n–k) = 1$$

$$=> k = n - 1$$

Time Complexity

Put $k = n - 1$ in eq-5...

$$T(n) = 2^{(n-1)}T(1) + 2^{(n-2)} + ... + 2^2 + 2^1 + 1$$

It is a Geometric Progression and the sum is...

$$T(n) = 1 \times \frac{(2^n - 1)}{(2 - 1)}$$

Time Complexity

Time complexity :

$$T(n) = O(2^n - 1) \approx O(2^n)$$

..... It is an Exponential Time Complexity :)

Binary Mapping

Do you find something special ??

Binary Mapping

Do you find something special ??

$$2^n - 1$$

is just 'n' number of 1's in binary representation

Binary Mapping

Do you find something special ??

$$2^n - 1$$

is just 'n' number of 1's in binary representation

1	2	...	$n-1$	n
1	1	...	1	1

(7)

Binary Mapping

The n disks can be mapped to those n bits

Binary Mapping

The n disks can be mapped to those n bits

The MSB corresponds to the largest disk

Binary Mapping

The n disks can be mapped to those n bits

The MSB corresponds to the largest disk

The LSB corresponds to the smallest disk

Binary Mapping

The n disks can be mapped to those n bits

The MSB corresponds to the largest disk

The LSB corresponds to the smallest disk

The other disks corresponds to the bits matching their order

Binary Mapping

The n disks can be mapped to those n bits

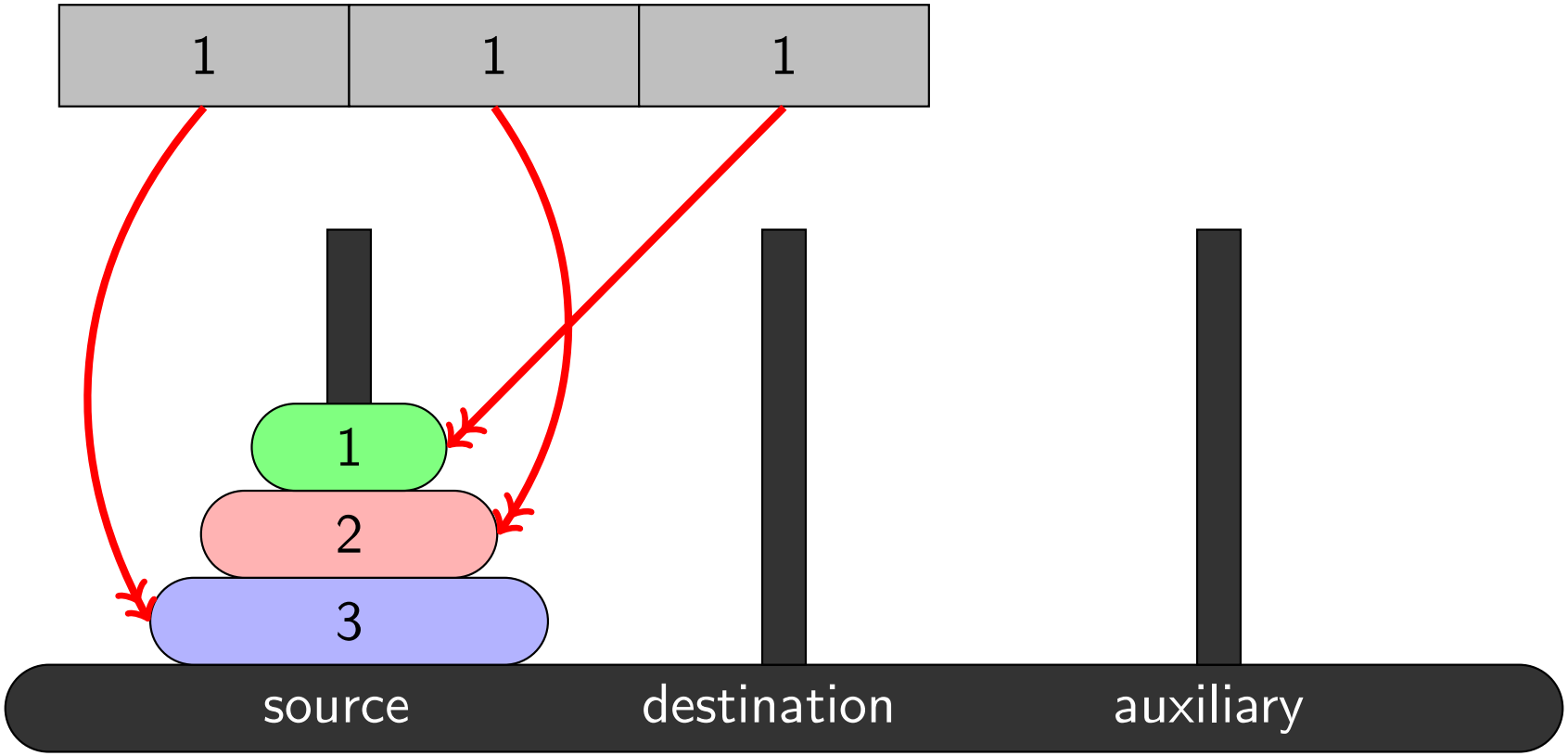
The MSB corresponds to the largest disk

The LSB corresponds to the smallest disk

The other disks corresponds to the bits matching their order

Let us see the binary maping for 3 disks

Binary Mapping



Binary Mapping

We want to minimize the time required to move the disk to destination

Binary Mapping

We want to minimize the time required to move the disk to destination

We will do it the following way

Binary Mapping

We want to minimize the time required to move the disk to destination

We will do it the following way

- we start from 000

Binary Mapping

We want to minimize the time required to move the disk to destination

We will do it the following way

- we start from 000
- And increment till 111

Binary Mapping

We want to minimize the time required to move the disk to destination

We will do it the following way

- we start from 000
- And increment till 111
- whenever a bit changes from 0 to 1 we move the corresponding disk

Binary Mapping

We want to minimize the time required to move the disk to destination

We will do it the following way

- we start from 000
- And increment till 111
- whenever a bit changes from 0 to 1 we move the corresponding disk
  - If the disk number is odd we always move it to the right

Binary Mapping

We want to minimize the time required to move the disk to destination

We will do it the following way

- we start from 000
- And increment till 111
- whenever a bit changes from 0 to 1 we move the corresponding disk
  - If the disk number is odd we always move it to the right
  - If the disk number is even we always move it to the left

Binary Mapping

But wait !!

Binary Mapping

But wait !!

There is no disk to the left of source pole

Binary Mapping

But wait !!

There is no disk to the left of source pole

No worries :)

Binary Mapping

But wait !!

There is no disk to the left of source pole

No worries :)

- We consider the disks to be in cyclic order

Binary Mapping

But wait !!

There is no disk to the left of source pole

No worries :)

- We consider the disks to be in cyclic order
- Thus auxiliary pole is on the left of source pole

Binary Mapping

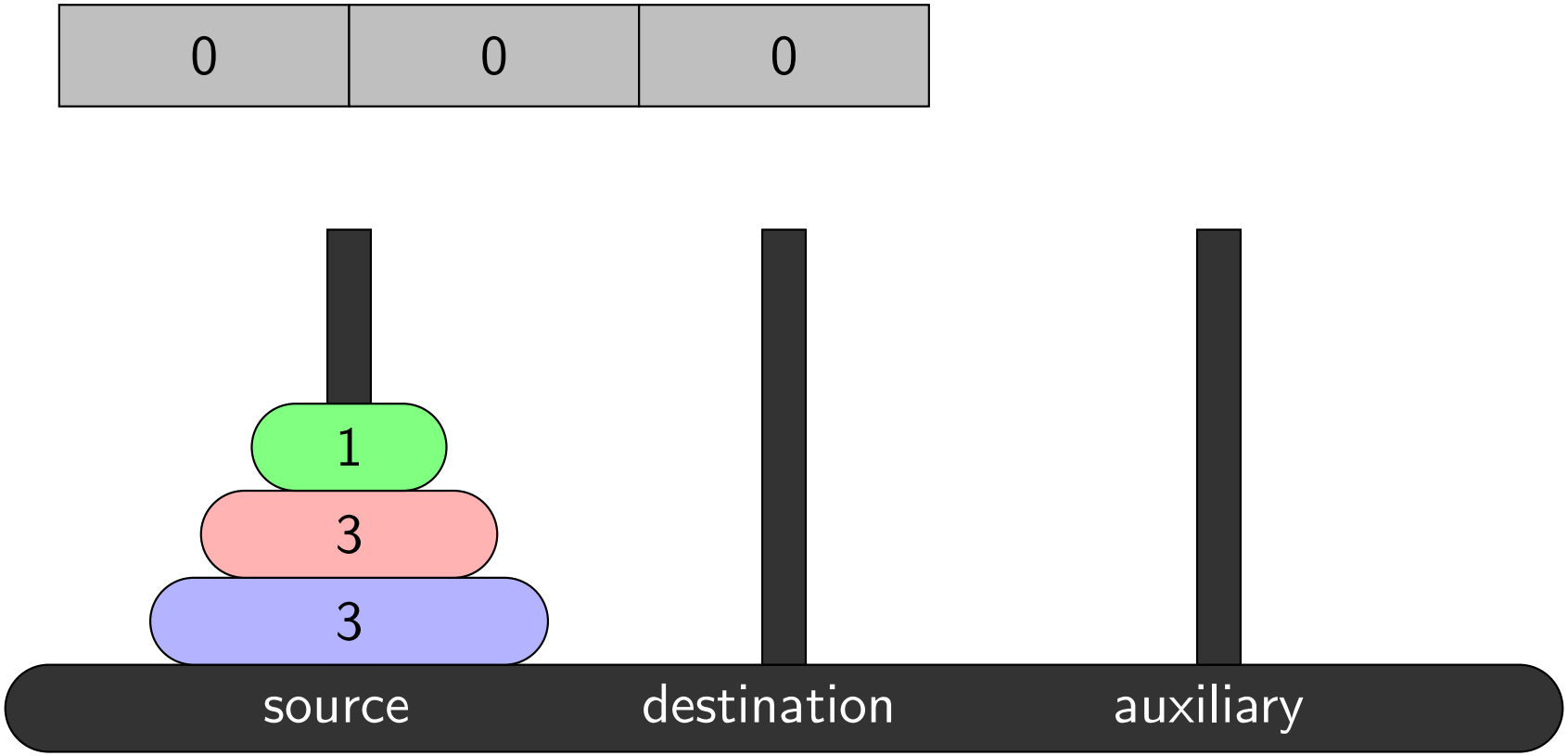
But wait !!

There is no disk to the left of source pole

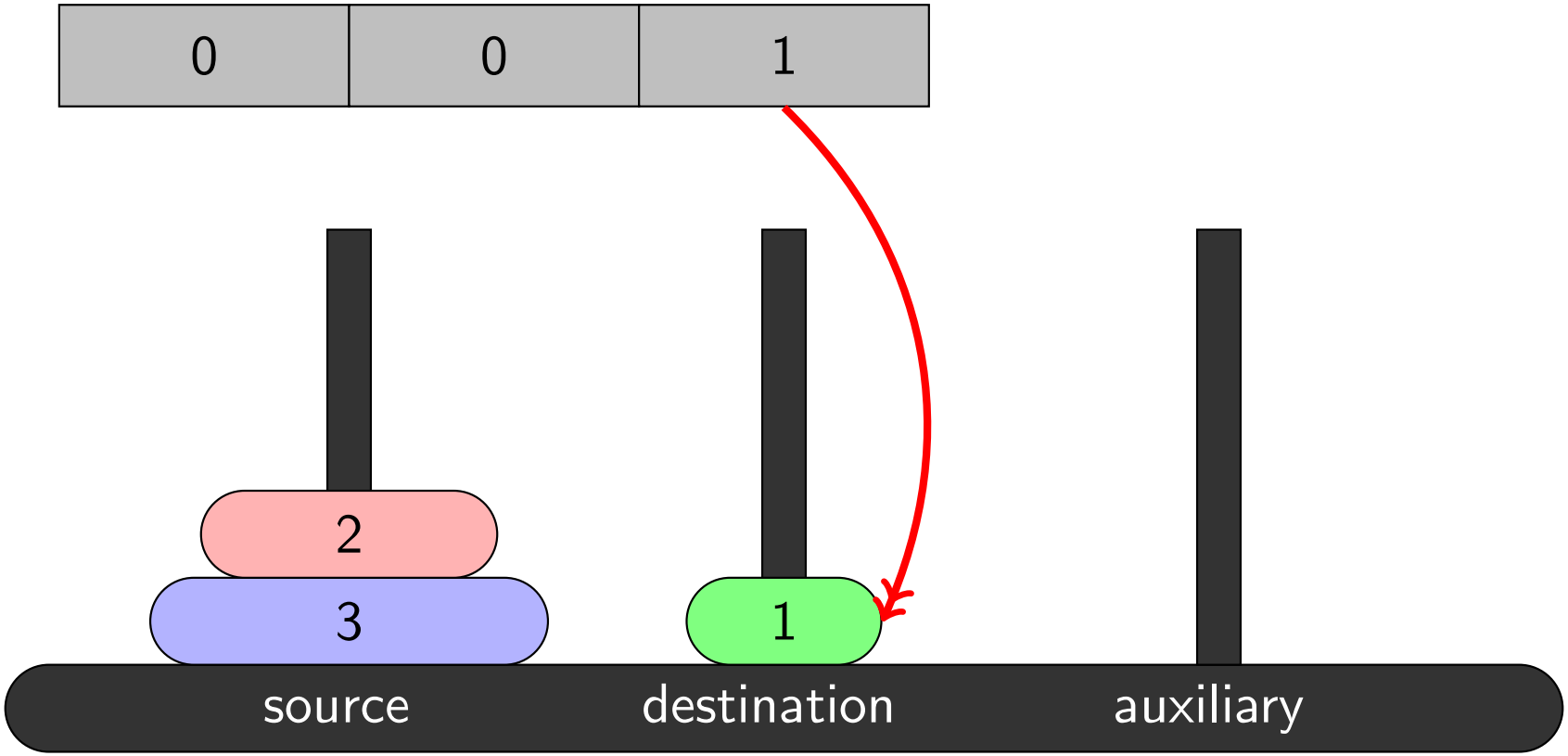
No worries :)

- We consider the disks to be in cyclic order
- Thus auxiliary pole is on the left of source pole
- And The source pole is on the right of auxiliary pole

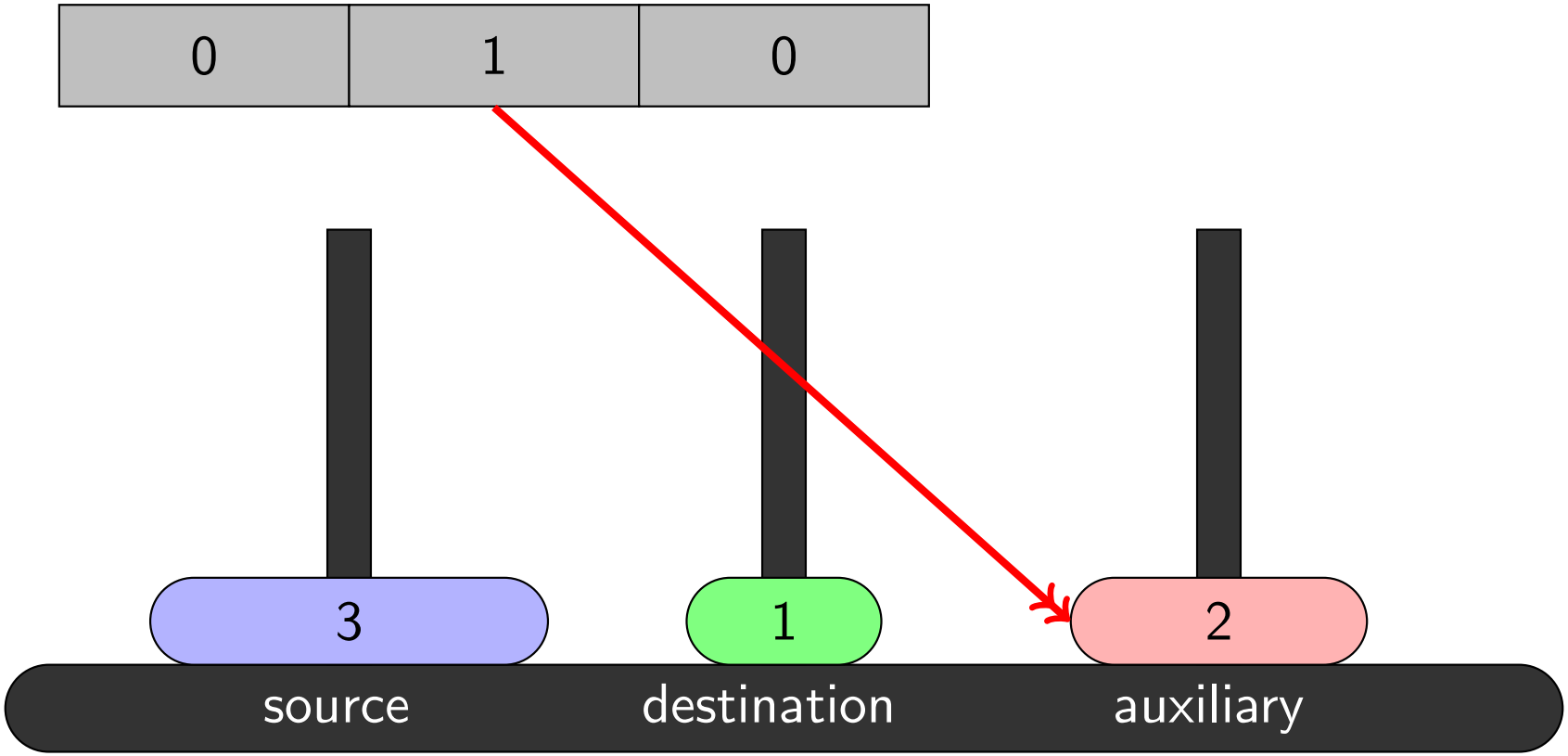
Binary Mapping



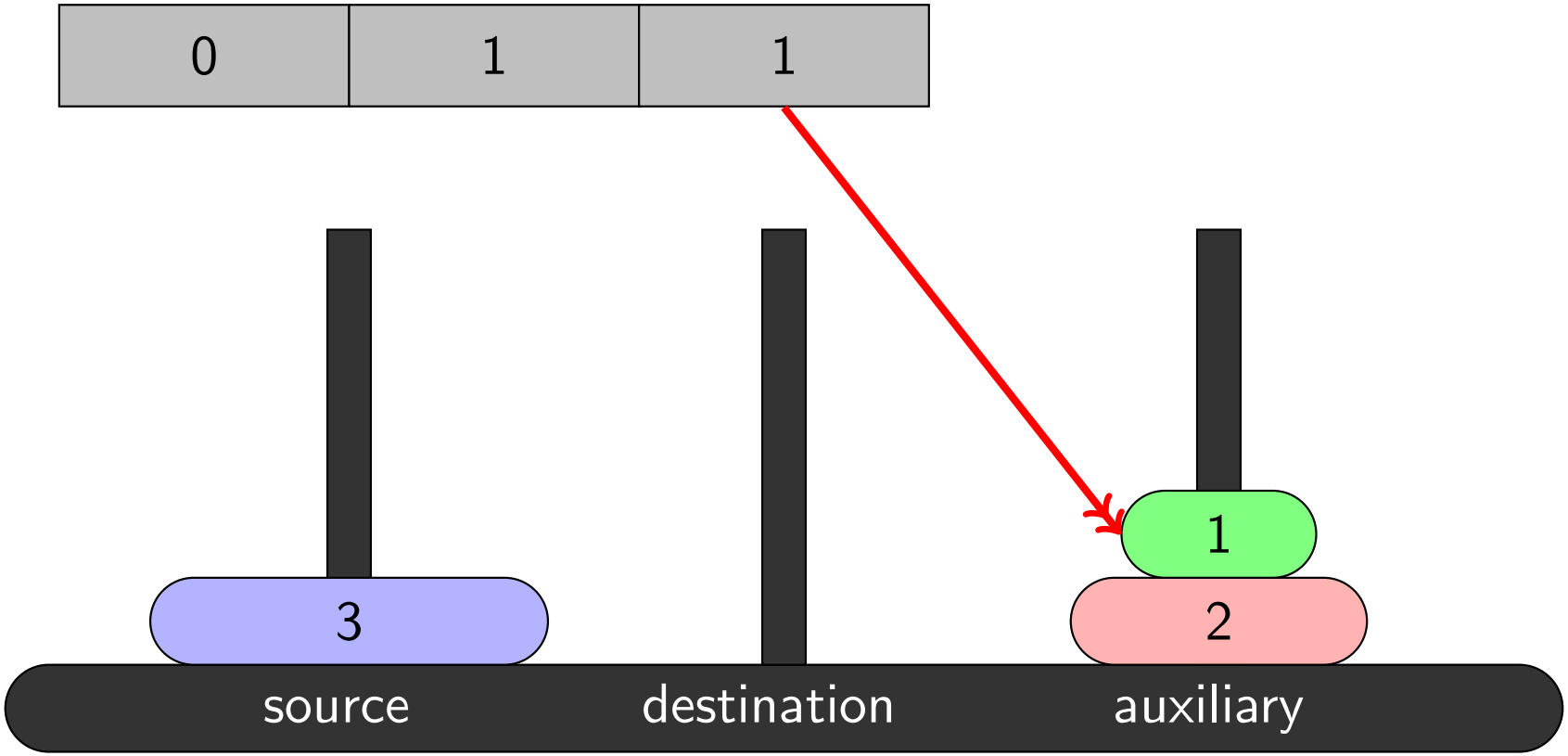
Binary Mapping



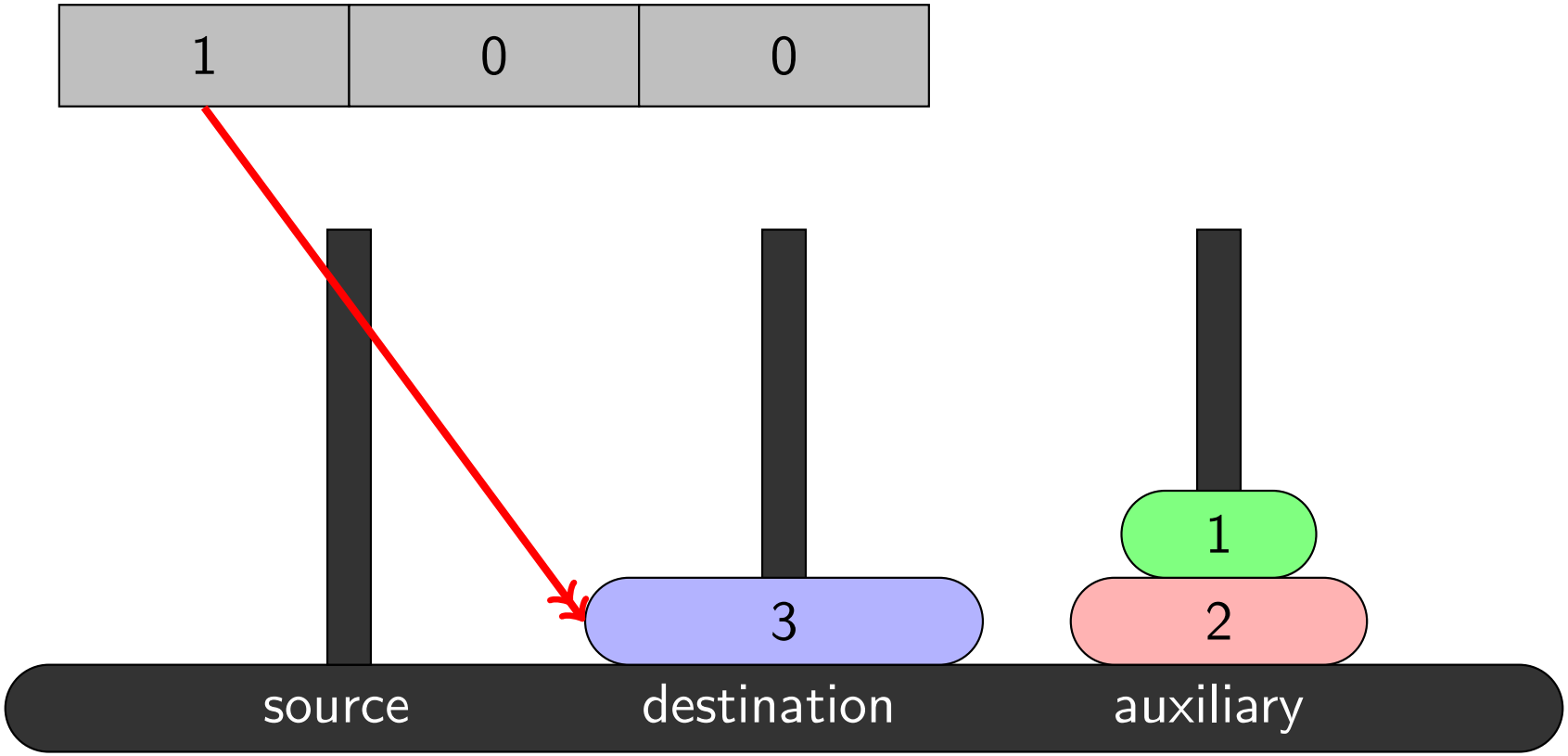
Binary Mapping



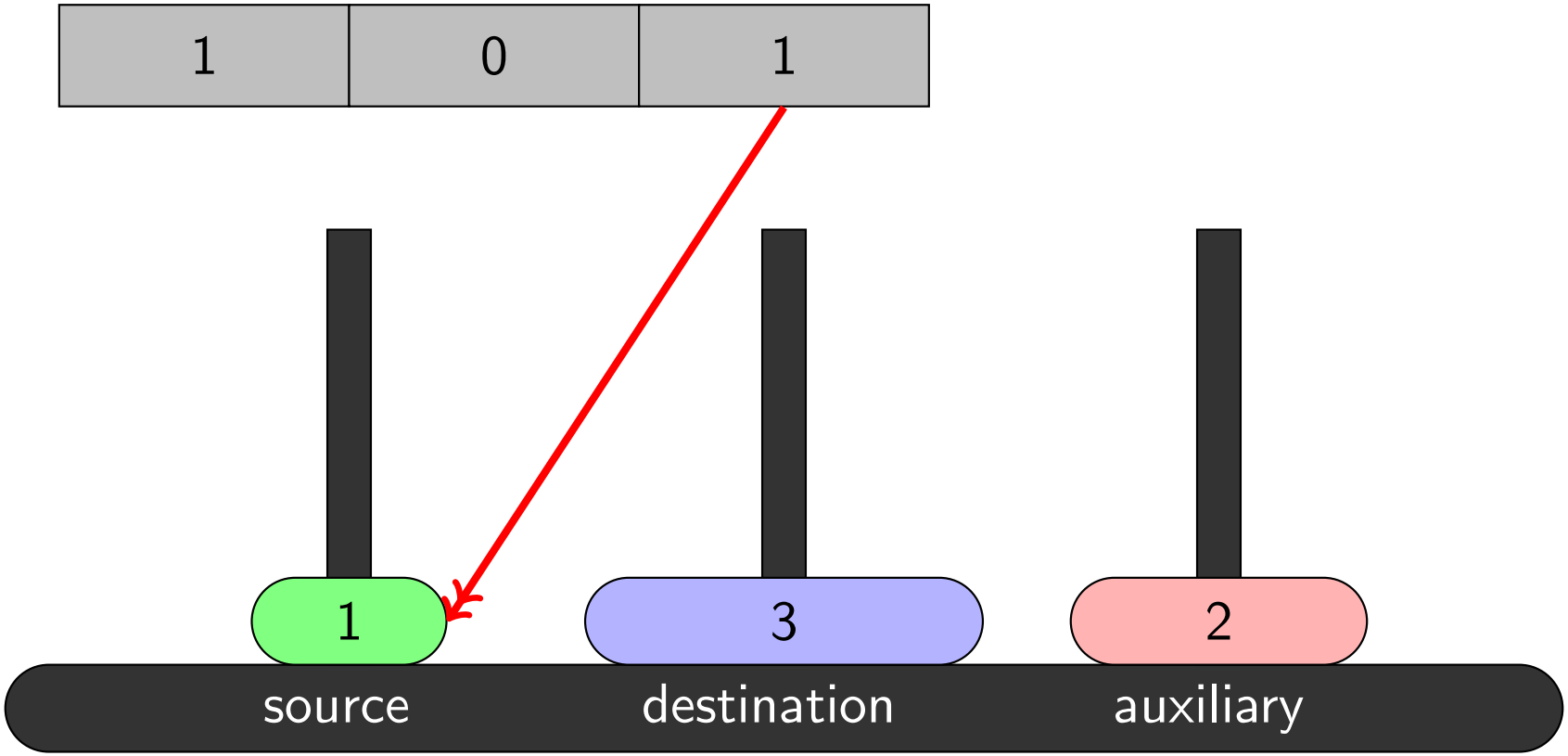
Binary Mapping



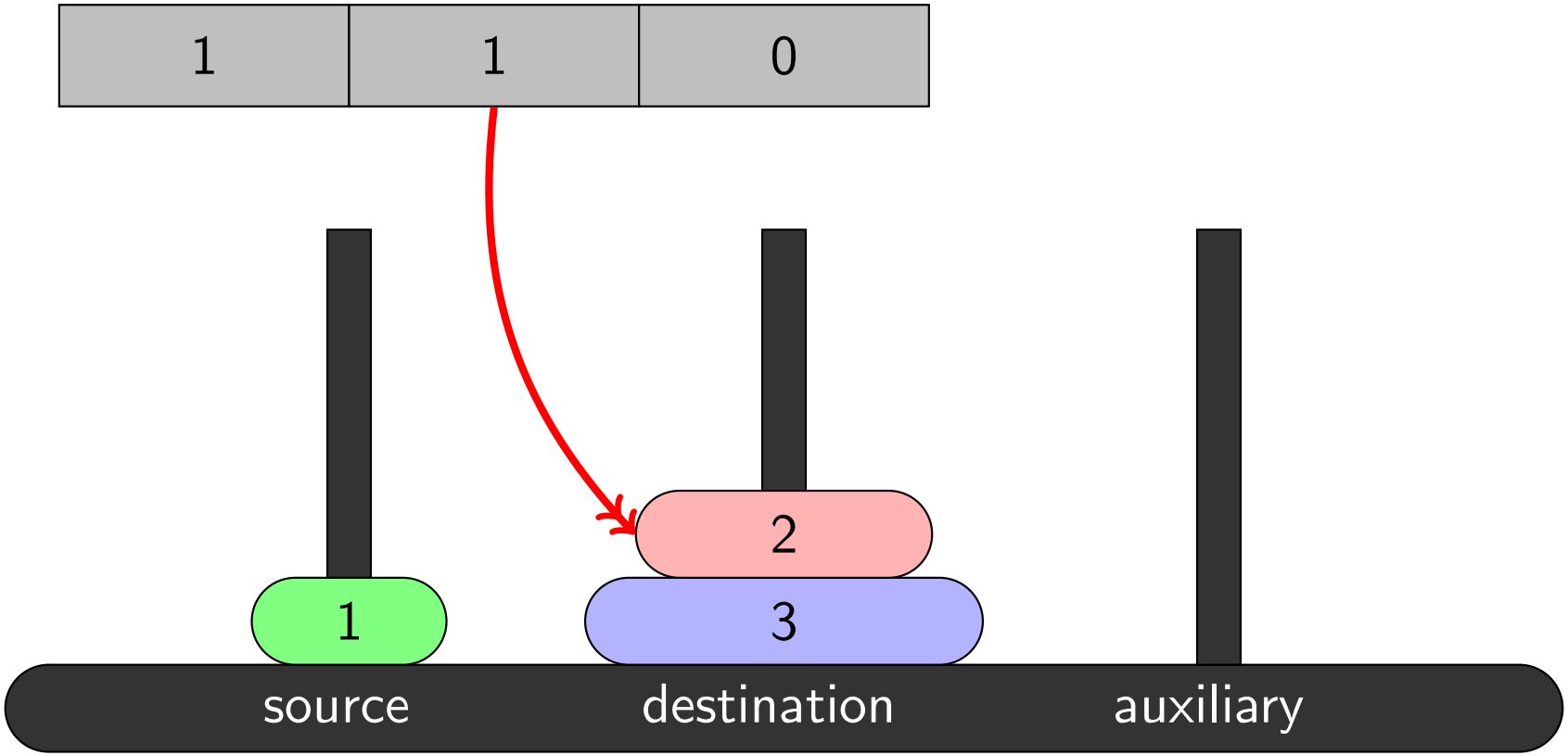
Binary Mapping



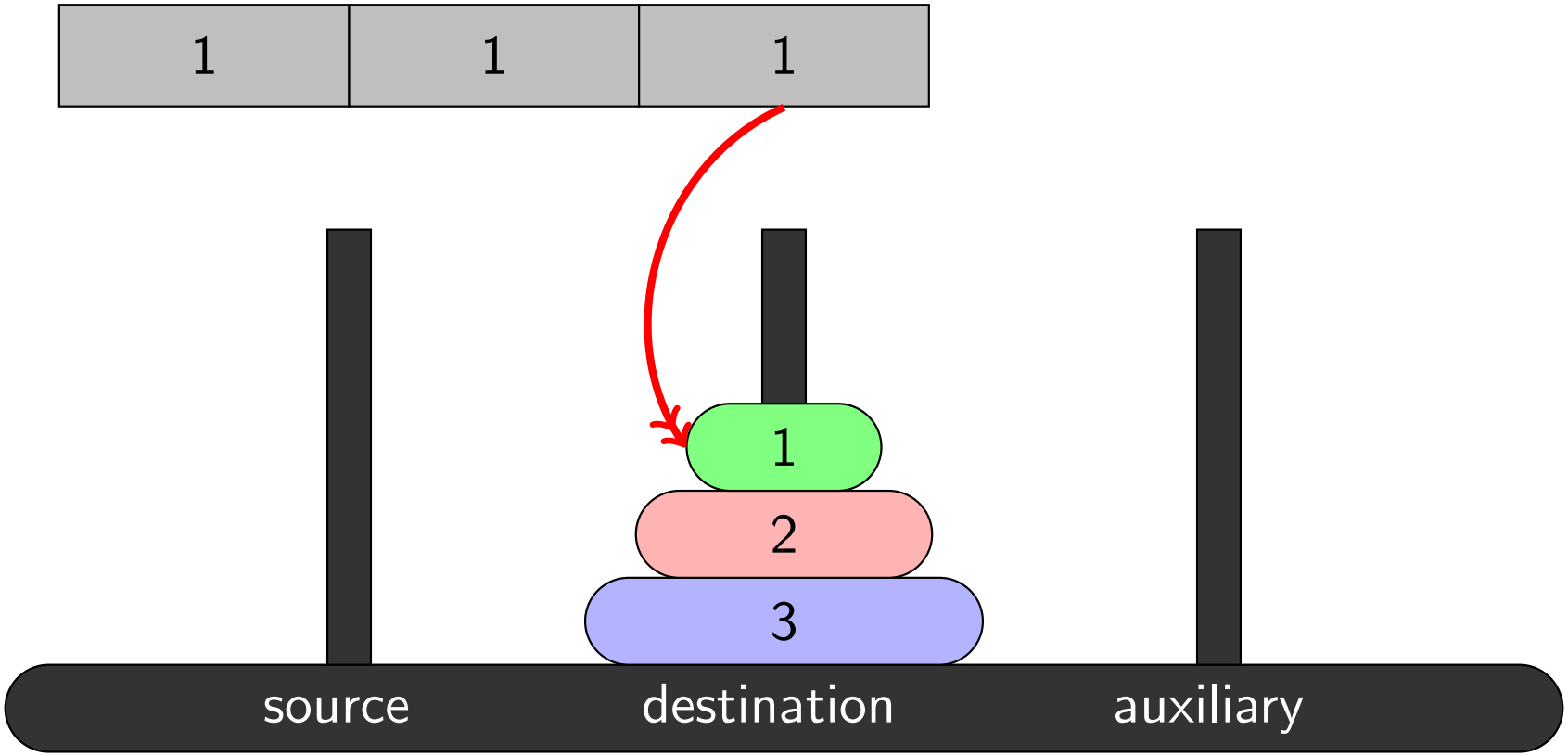
Binary Mapping



Binary Mapping



Binary Mapping



Binary Mapping

If moving 1 disk takes 1 second

Binary Mapping

If moving 1 disk takes 1 second

$$2^{64}$$

seconds

Binary Mapping

If moving 1 disk takes 1 second

2^{64}

seconds

- That is nearly

580

billion years